6COSC023W

Computer Science Final Project

Final Year Project (FYP) – Report

ENDYMION –

A Data-Driven Framework for Assessing Lunar Terrain Hazards and Supporting Hazard-Aware Navigation

Student: Yair Cortes Torres (W20475998)

Supervisor: Dr. Alan Immanuel Benjamin Vallavaraj

Degree: BSC (HONS) COMPUTER SCIENCE

This report is submitted in partial fulfilment of the requirements for the

BSc (Hons) Computer Science degree

BEng Software Engineering degree

School of Computer Science & Engineering

University of Westminster

Date: 26/04/2026

Document Scope

The purpose of this document is to describe and reflect on the processes that took place in developing the Final Project. Discuss any ethical issues associated with your project and describe the methodology that was adopted to develop its design, implementation and testing.

All chapter word counts in this document are approximate and are not intended to be prescriptive.

Declaration

This report has been prepared based on my own work. Where other published and unpublished source materials have been used, these have been acknowledged in references.

Word Count: 14252

Student Name: Yair Aien Cortes Torres

Date of Submission: 26/04/2026

Abstract

Lunar south-polar exploration presents a major challenge for autonomous navigation because extreme topography, low-angle illumination, and permanently shadowed regions make terrain interpretation and safe route planning especially difficult. At the same time, these regions are of high scientific and strategic interest due to their potential to preserve water ice and other volatiles. This project presents **Endymion**, a modular, reproducible, and data-driven framework for lunar terrain hazard assessment and hazard-aware navigation focused on the Moon's south polar region. Rather than treating terrain analysis, hazard modelling, and route generation as separate tasks, the system integrates them into one structured pipeline capable of transforming lunar terrain data into explicit hazard representations and safer navigation outputs.

The principal aim of the project was to design and implement a system that combines terrain analysis, crater-aware hazard modelling, and safety-weighted pathfinding within one integrated analytical pipeline. The methodology followed an iterative, prototype-led lifecycle.

An early terrain-only prototype was first constructed to validate the terrain-to-route chain, after which the system was refactored into a cleaner modular pipeline with stable artefact reuse, explicit module responsibilities, and a fixed comparison contract for benchmarking. South-polar digital elevation data formed the basis of the workflow, from which terrain descriptors such as slope and RMS roughness were derived. These features were then fused into a hazard surface, converted into a traversal-cost grid, and used by a corridor-aware Weighted A* pathfinding stage to compute safer routes under constrained search conditions.

The final system extended beyond the terrain-only baseline by incorporating crater-aware products and a baseline machine-learning stage. Crater presence, crater proximity, and local crater density were represented as ROI-aligned raster products and integrated into the hazard model. In addition, a Random Forest classifier was introduced as a supervised baseline to generate crater-probability outputs from cached terrain-derived features. This allowed the system to compare terrain-only, crater-aware, and ML-assisted hazard variants under the same reproducible framework. The main technologies used in the project included Python, NumPy, Pandas, Rasterio, Scikit-learn, Google Colab, and Visual Studio Code, with GitHub and Notion supporting version control and project organisation.

Testing and evaluation were carried out through component-level verification, end-to-end pipeline validation, and controlled benchmark comparison. The final results showed that Endymion successfully generated hazard-aware routes from real lunar south-polar terrain data and supported fair comparison across multiple hazard-model variants. Under the tested benchmark conditions, the terrain-only baseline remained the strongest overall model, while the crater-aware and ML-assisted variants demonstrated that richer hazard signals could be integrated successfully without compromising reproducibility or evaluation consistency.

In conclusion, the project delivered a functioning and academically defensible hazard-navigation framework rather than a one-off experimental prototype. Although Endymion is not yet a mission-ready navigation system, it provides a structured and extensible basis for future work. Recommended next steps include illumination-aware hazard modelling, broader validation across additional lunar regions, refinement of the crater-prediction stage, and further extension of the system towards larger-scale lunar traversal analysis.

Acknowledgements

I would like to extend my gratitude to my project supervisor, lecturers, and colleagues within the School of Computer Science and Engineering for their invaluable guidance and support. Additionally, I wish to express my heartfelt appreciation to my family and friends for their steadfast support and understanding throughout the duration of this paper.

Table of Contents

Document Scope 1
Declaration 2
Abstract 3
Acknowledgements 4
Table of Contents 5
List of Figures 7
List of Tables 7
1. Introduction 8
    1.1 Problem Statement 8
    1.2 Aims and Objectives 9
2. Background 10
    2.1 Literature Review 10
    2.2 Review of Projects/Applications (Gap Analysis) 12
    2.3 Review of tools, frameworks and techniques 14
3. Legal, social, sustainability and ethical issues 14
    3.1 Legal Issues 14
    3.2 Social and Professional Issues 14
    3.3 Sustainability Issues 15
    3.4 Ethical Issues 15
    3.5 Security Issues 15
4. Methodology 16
    4.1 Lifecycle approach 16
    4.2 Project phases and milestones 17
    4.3 Testing strategy 18
5. Design 19
    5.1 System architecture 19
    5.2 Data flow and module responsibilities 20
    5.3 Hazard model design 21
    5.4 Pathfinding design 22
    5.5 UML & DFD Discussion 23
6. Tools and Implementation 24
    6.1 Tools 25
    6.2 Implementation 26
7. Testing and Results 35

7.1 Test Coverage 35
7.2 Test methodology 39
8. Conclusions and Reflections 42
References 45
Bibliography 46
Appendix 47

List of Figures

FIGURE 1 - SYSTEM'S FLOW 19
FIGURE 2 - FINAL ENDYMION PIPELINE ARCHITECTURE SHOWING THE CORE MODULES, SAVED ARTEFACTS, AND EXTENSION PATHS FOR BENCHMARKING AND ML-ASSISTED PROCESSING. 20
FIGURE 3 - DATA FLOW DIAGRAM SHOWING THE SAVED ARTEFACTS EXCHANGED BETWEEN MODULES IN THE FINAL PIPELINE. 21
FIGURE 4 - HAZARD FUSION LOGIC IN THE FINAL ENDYMION HAZARD MODEL 22
FIGURE 5 - FINAL UML-STYLE COMPONENT DIAGRAM OF THE IMPLEMENTED ENDYMION MODULE STRUCTURE. 24
FIGURE 6 - PROTOTYPE PIPELINE OVERVIEW (DEM → FEATUREEXTRACTOR → HAZARDASSESSOR → COST GRID → PATHFINDER) 26
FIGURE 7 - PATH OVERLAY ON DEM SHOWING THE TERRAIN-ONLY PROTOTYPE RESULT 28
FIGURE 8 - ENDYMION'S PIPELINE FLOW 29
FIGURE 9 - PHASE 1 VS PHASE 2 BENCHMARK COMPARISON 29
FIGURE 10 - ML BRIDGE FLOW 34
FIGURE 11 - ENDYMION CLI RUN SUMMARY 34
FIGURE 12 - VALIDATION APPROACH 39
FIGURE 13 - BENCHMARK PROCESS 41

List of Tables

TABLE 1 - GAP ANALYSIS SUMMARY 13
TABLE 2 - FUNCTIONAL REQUIREMENTS 35
TABLE 3 - NON-FUNCTIONAL REQUIREMENTS 37
TABLE 4 - TEST CASE SUMMARY 39
TABLE 5 - SUMMARISES THE BENCHMARK ELEMENTS 40
TABLE 6 - SUMMARISES THE STRUCTURE OF THE BENCHMARK SET. 40

1. Introduction

The lunar south pole has become one of the most important targets for contemporary planetary exploration because of its scientific value, its relevance to future sustained surface operations, and the unusual environmental conditions that distinguish it from more accessible lunar terrains. Permanently Shadowed Regions (PSRs) are now known to contain or preserve volatile materials, including water ice, making them strategically important for both scientific investigation and long-term exploration planning (Li et al., 2018; McClanahan et al., 2024). At the same time, these regions are difficult to characterise and even more difficult to traverse safely, because extreme topography, persistent darkness, and limited visibility complicate terrain assessment and navigation (Bickel et al., 2021; Implementation Plan for a NASA Integrated Lunar Science Strategy in the Artemis Era, 2024).

Endymion was developed in response to this problem. The project is a modular data-driven framework that transforms south-polar terrain data into explicit hazard-aware navigation outputs through a reproducible analytical pipeline. Rather than treating terrain analysis, hazard assessment, and route generation as separate tasks, Endymion integrates them into a single reproducible pipeline. In doing so, it addresses a central problem in lunar surface operations: how to derive a safer and more interpretable route through terrain that is both physically hazardous and observationally incomplete.

1.1 Problem Statement

The lunar south pole represents one of the most complex and challenging environments for autonomous navigation within the entire Solar System. This region is characterised by steep crater walls, a rugged and highly variable local topography, intermittent illumination due to the Moon's axial tilt, and the presence of extensive permanently shadowed regions (PSRs). These PSRs, which can remain in darkness for geologically significant periods, pose considerable obstacles to navigation, as they limit the availability of solar power and complicate sensor operations. Collectively, these harsh conditions dramatically reduce the reliability of conventional visual sensing techniques and make the interpretation of the terrain significantly more difficult, even when high-resolution orbital data are accessible (Li et al., 2018; Bickel et al., 2021).

A significant technical challenge in this context is that the lunar datasets available do not provide an out-of-the-box navigation solution. The digital elevation models (DEMs) used for lunar topographic analysis are derived from a variety of sources and require comprehensive transformation into analysis-ready terrain representations to be effectively utilised for hazard assessment. The Lunar Orbiter Laser Altimeter (LOLA) serves as the primary source of elevation data, forming the backbone of lunar topographical analysis. To enhance the detail and usability of DEM products, additional processing and data fusion are necessary to maintain important small-scale terrain characteristics that are crucial for safe traversal (Smith et al., 2009; Barker et al., 2016). Consequently, any rover or mission planning framework cannot simply rely on the raw elevation data; it must translate this terrain information into meaningful risk assessments, which include parameters like slope, surface roughness, proximity to craters, and overall safe traversability.

This challenge becomes even more critical in the context of the current Artemis missions, where the success of operations depends not just on reaching scientifically significant areas but also on navigating there safely and efficiently, given the demanding environmental constraints present on

the lunar surface (Implementation Plan for a NASA Integrated Lunar Science Strategy in the Artemis Era, 2024). Current methodologies often focus narrowly on specific aspects of the problem: for instance, some are dedicated solely to mapping initiatives, others specialise in detecting craters, while still others concentrate on planning effective paths. However, very few systems integrate these elements into a cohesive framework capable of supporting hazard-aware navigation that is both reproducible and applicable over a consistent and rigorously testable terrain context.

To address this crucial gap in the existing approaches, the Endymion project has emerged, combining terrain-derived analyses with crater-aware hazard representation and constrained route planning all within a singular, integrated system. While this project is specifically tailored toward exploring the lunar south pole, the underlying methodological challenges it tackles are relevant in a broader sense, extending to other environments where safe navigation must be executed despite low visibility, limited data availability, or highly irregular and rugged terrain conditions.

1.2 Aims and Objectives

The overall aim of Endymion is to develop a modular, data-driven framework for terrain hazard assessment and hazard-aware navigation in lunar polar regions. The project seeks to combine terrain analysis, crater-aware hazard modelling, and safety-weighted pathfinding within a single reproducible pipeline, with particular focus on south-polar terrain where topographic and illumination-related constraints make route planning especially difficult.

To achieve this aim, the following objectives were defined:

Data ingestion and standardisation.

To design a DataHandler capable of loading, validating, and standardising south-polar lunar elevation data and associated crater information into analysis-ready formats.

Terrain feature engineering.

To implement a FeatureExtractor that derives terrain descriptors, principally slope and RMS roughness, from DEM patches in a form suitable for later hazard modelling.

Hazard modelling.

To develop a HazardAssessor that transforms terrain and crater-related signals into a unified hazard surface that can be interpreted directly for navigation.

Risk-aware pathfinding.

To implement a Pathfinder based on Weighted A* over a hazard-derived traversal-cost grid, allowing safe route computation under fixed search constraints.

Performance evaluation.

To create an Evaluator and BenchmarkRunner capable of assessing route quality, hazard exposure, search effort, and comparative behaviour across model variants.

Machine-learning extension.

To introduce a baseline supervised machine-learning component capable of generating an ROI-aligned crater-probability product from cached terrain-derived features, without breaking the wider reproducibility, storage, and benchmarking contract of the system

Together, these objectives define Endymion not simply as a terrain-visualisation project, but as a structured hazard-navigation framework whose outputs can be inspected, tested, and compared under controlled conditions.

2. Background

The Lunar Polar Regions, particularly the south pole, present one of the most challenging yet scientifically rewarding environments within our Solar System. These regions are characterised by a diverse and extreme topography, which includes deep craters, rugged terrain, and vast expanses of icy deposits. The unique lighting conditions, with low-angle illumination, further complicate the landscape, making it difficult to navigate and traverse. Moreover, there are Permanently Shadowed Regions (PSRs) that remain devoid of sunlight, creating areas that are permanently cloaked in darkness. These PSRs are of particular interest due to their potential to harbour water ice and other volatile compounds; however, they also pose significant obstacles for exploration missions.

Current navigation systems often face considerable challenges when operating in these lunar environments. The extreme lack of reliable visual data, intensified by the absence of natural light in PSRs, means that traditional methods of hazard identification become nearly impossible. Without sufficient visibility, identifying and navigating around potential hazards, such as steep terrain or rocky outcrops, becomes a complex task that can lead to mission failures.

The Endymion project seeks to address these critical gaps in navigation and exploration capabilities by integrating several advanced technologies into a unified framework. This initiative combines remote-sensing data analysis to map the terrain accurately, machine learning algorithms for predicting hazards, and safety-weighted pathfinding techniques that optimise routes while minimising risks.

By bringing these elements together, the Endymion project aims to enhance the safety and effectiveness of both robotic and human missions to the Lunar Polar Regions, enabling us to unlock the scientific secrets that these remote areas hold.

2.1 Literature Review

2.1.1 The lunar polar environment and volatile significance

Scientific interest in the lunar south pole has increased significantly because PSRs may contain preserved volatiles, especially water ice. Li et al. (2018) provided direct evidence of surface-exposed water ice in the lunar polar regions, while McClanahan et al. (2024) presented evidence for widespread hydrogen sequestration in south-polar cold traps. These findings have made the polar regions especially important for future mission planning, because they connect scientific discovery with the practical long-term possibility of in-situ resource use.

However, the same regions are extremely difficult to study and traverse. Their environmental conditions include poor illumination, low-angle solar incidence, deep shadow, and highly irregular cratered terrain. Bickel et al. (2021) showed that deep learning can help reveal geological structure inside PSRs, which highlights both the importance and the difficulty of interpreting these

environments. In other words, the South Pole is not just scientifically relevant; it is also a terrain-analysis problem in which data quality and visibility directly affect operational safety.

2.1.2 Lunar topography and DEM foundations

Effective hazard analysis begins with an accurate and detailed representation of terrain. When it comes to lunar exploration, the Lunar Orbiter Laser Altimeter (LOLA) serves as the crucial geodetic and elevation backbone. Smith et al. (2009) highlight the importance of LOLA as an indispensable source of topographic measurements during the Lunar Reconnaissance Orbiter mission, laying the groundwork for subsequent advancements in gridded elevation analysis.

Building upon this foundational work, Barker et al. (2016) made significant strides in the generation of Digital Elevation Models (DEMs) by integrating data from LOLA with imagery from the SELENE Terrain Camera. This combination not only enhanced the quality of elevation data but also ensured that finer-scale topographic features were preserved, which is essential for analysing navigation and traversal challenges on the lunar surface.

This nuanced understanding of terrain becomes particularly critical in the context of Endymion. A robust hazard-navigation framework cannot merely rely on broad elevation data; it must incorporate detailed terrain products that capture local nuances. These variations are vital for assessing factors such as slope, surface roughness, the contours of crater margins, and other risk indicators that play a significant role in safe navigation.

Adding weight to this assertion, Deutsch et al. (2020) demonstrate that the roughness characteristics of circumferential lunar craters are both significant and highly variable. This highlights the necessity of utilising derived terrain descriptors as integral components of hazard assessment, underscoring the need for precise terrain representation in mitigating risks during lunar exploration.

2.1.3 Crater detection and machine learning context

As lunar terrain analysis becomes more detailed, crater detection has increasingly moved toward machine-learning and deep-learning approaches. Bickel et al. (2021) demonstrated that ML techniques can recover information in low-visibility polar contexts, while Giannakis et al. (2024) proposed a flexible crater-detection approach using the Segment Anything Model. Sinha and Paul (2025) further review deep-learning approaches for automated crater detection and show that crater mapping is now a mature area of lunar image analysis.

These studies are relevant to Endymion, but they also highlight a difference in project emphasis. Much of the crater-detection literature is concerned with recognition accuracy or large-scale crater inventory generation. Endymion uses crater-related prediction more narrowly: not as an end in itself, but as an additional hazard cue that can be fused into route planning. This makes crater prediction operational rather than purely descriptive in the context of the project.

2.1.4 Hazard-aware pathfinding

Once terrain and crater information have been transformed into hazard signals, the next problem is how to navigate through them. For this, graph-search methods remain a natural foundation. Pohl (1970) formalised heuristic pathfinding in graph space and provides a suitable theoretical basis for Weighted A* style search. In Endymion, that logic is adapted into a hazard-aware traversal framework in which hazardous cells become more expensive or effectively blocked, and the planner searches for a safe path rather than simply the shortest geometric route.

The significance of this shift is methodological. In planetary navigation, pathfinding is only as good as the terrain-risk model it depends on. A shortest route across unsafe terrain is not operationally useful. Endymion therefore draws from both hazard modelling and pathfinding literature to frame navigation as a constrained safety problem rather than a purely geometric one.

2.2 Review of Projects/Applications (Gap Analysis)

A range of existing projects, datasets, and research tools are relevant to Endymion, but most of them address only part of the wider problem of lunar south-polar hazard-aware navigation. Some provide authoritative terrain data, some improve visibility inside PSRs, some focus on crater detection, and others provide general pathfinding methods. However, fewer approaches combine these elements into a single reproducible analytical framework that can support controlled hazard comparison and route evaluation. This is the main gap that Endymion was designed to address.

At the data level, the lunar community already benefits from high-quality terrain products. LOLA remains the principal elevation backbone for lunar topography, while later DEM work has improved the representation of local terrain detail through fused elevation products (Smith et al., 2009; Barker et al., 2016). These resources are essential, but they are not in themselves navigation systems. They provide terrain information, not an operational interpretation of hazard. In practice, a rover-navigation framework still has to transform those datasets into aligned local products, derive meaningful risk indicators, and connect them to route selection. Endymion addresses this by turning south-polar DEM inputs into a consistent ROI-based terrain workflow through the DataHandler and FeatureExtractor, rather than treating the source data as already navigation-ready.

A second area of relevant prior work concerns the observation of PSRs and other low-visibility polar surfaces. Studies such as Bickel et al. (2021) show that deep learning can recover geological information from environments that are extremely difficult to observe with conventional approaches. This is important because it confirms that polar darkness is not merely a visual inconvenience, but a genuine sensing and interpretation problem. However, such studies are usually focused on improving visibility or recovering terrain structure, rather than producing a complete hazard-to-navigation framework. Endymion builds on this problem setting, but shifts the focus from image enhancement to hazard-aware route reasoning over derived terrain and crater products.

A third area is crater detection and crater mapping. Recent literature shows strong progress in automated crater recognition, including flexible segmentation and deep-learning-based methods (Giannakis et al., 2024; Sinha and Paul, 2025). These approaches are highly relevant because crater morphology is one of the most important hazard cues in the south-polar environment. Nevertheless, most crater-detection work is designed either for scientific inventory generation or for image-based recognition performance. The operational question addressed by Endymion is narrower and more system-oriented: how crater-related information can be converted into a hazard signal that influences route planning under fixed benchmark conditions. In this sense, Endymion does not compete with the crater-detection literature directly; instead, it uses crater-awareness as one part of a wider navigability framework.

A fourth area concerns pathfinding itself. Graph-search methods such as A* and Weighted A* are well established, and their theoretical basis is strong (Pohl, 1970). However, generic pathfinding

theory does not solve the lunar hazard problem on its own. A planner is only as useful as the cost model that drives it. If terrain steepness, surface roughness, crater proximity, or blocked regions are not represented properly, then even an efficient planner may simply compute a geometrically short but operationally unsafe route. The gap here is therefore not in the lack of pathfinding algorithms, but in the lack of a reproducible hazard formulation tied closely enough to terrain products for route outputs to be meaningfully evaluated.

From this review, three main gaps become clear. First, there is a gap between scientific terrain products and navigation-ready hazard surfaces. Second, there is a gap between crater recognition and route-planning utility, since many crater studies stop at detection rather than integration into traversability analysis. Third, there is a gap between isolated experimentation and controlled comparative evaluation, because many workflows are not designed around fixed benchmark cases, stable saved artefacts, or direct comparison between hazard-model variants.

Endymion was developed to sit in that gap. Its contribution is not simply a new DEM, a new crater detector, or a new graph-search algorithm. Instead, it is the integration of terrain processing, crater-aware hazard modelling, pathfinding, and evaluation into one modular and reproducible framework. This is why the project's value is architectural and methodological as much as algorithmic:

It establishes a way to test how different hazard representations alter navigation behaviour under a common execution contract.

That systems-level focus is what differentiates Endymion from many of the individual approaches on which it builds.

2.2.1 Gap analysis summary

Aspect	Existing approaches	Limitation	Endymion response
Terrain data	High-quality lunar DEM and LOLA-based products	Terrain data are not directly navigation-ready	Converts DEM data into ROI-aligned terrain products for hazard use
PSR observation	Image enhancement and shadow-region interpretation	Often focused on visibility recovery rather than navigation	Treats PSR difficulty as a hazard-analysis and route-planning problem
Crater detection	Strong ML and segmentation-based crater mapping literature	Detection does not automatically support traversability decisions	Uses crater-awareness as a risk term inside hazard fusion
Pathfinding	Established A* / Weighted A* theory	Planner quality depends on hazard representation	Couples planner to hazard-derived cost and controlled route evaluation
Evaluation	Many studies report local performance only	Fair cross-model comparison is often limited	Uses fixed ROI, fixed cases, and fixed planning settings for benchmarking

Table 1 - Gap Analysis Summary

2.3 Review of tools, frameworks and techniques

The development of Endymion utilises a specialised stack of geospatial and machine learning libraries to handle planetary data efficiently within a cloud-based environment.

- **Python 3.10+:** Selected as the primary language for its extensive ecosystem of scientific libraries and its suitability for rapid prototyping of complex data pipelines.
- **Google Colab:** Provides the necessary high-performance CPU and GPU access to manage large-scale lunar datasets and train the Random Forest predictor without local hardware constraints.
- **Geospatial Libraries:** *Rasterio* and *GDAL (Geospatial Data Abstraction Library)* are used for reading PDS-compatible *GeoTIFFs* and managing coordinate transformations (South Polar Stereographic), which is essential for accurate ROI extraction.
- **Machine Learning (Scikit-Learn):** Used for the implementation of the *Random Forest* model. This technique was selected over CNNs for the prototype phase due to its lower computational cost, interpretability, and ability to handle tabular raster features effectively.
- **Algorithmic Frameworks:** *NumPy* handles the high-speed numerical computation of terrain gradients, while *NetworkX* and custom *A** implementations facilitate hazard-weighted pathfinding.

3. Legal, social, sustainability and ethical issues

This section examines the legal, social, professional, sustainability, and ethical considerations surrounding the development and deployment of the Endymion prototype. As a system designed for high-risk, safety-critical autonomous navigation in space exploration, Endymion must adhere to rigorous standards of data integrity, scientific ethics, and professional responsibility.

3.1 Legal Issues

The legal considerations for Endymion primarily involve data provenance and intellectual property. All data utilised in this project, including the LOLA gridded DEMs and the Robbins Crater Catalogue, are open-source scientific datasets obtained from NASA's public repositories, such as the *Planetary Data System* (PDS). These datasets are provided for public scientific use; however, strict adherence to citation and attribution standards is legally and professionally required to ensure scientific integrity.

3.2 Social and Professional Issues

From a social perspective, Endymion contributes to the broader human endeavour of space exploration. By addressing the navigational challenges of the lunar south pole, the project supports international programs such as NASA's Artemis and ESA's Argonaut missions. Professionally, the system must be developed following the "best practices for scientific computing" to ensure results are reproducible and verifiable. Because this system is intended for "safety-critical" navigation, there is a professional obligation to ensure that hazard maps are accurate to prevent vehicle immobilisation or damage, which could lead to mission failure and significant financial loss.

3.3 Sustainability Issues

Sustainability in this project is twofold: environmental and operational.

- **Operational Sustainability:** Endymion directly supports the search for frozen water deposits and hydrogen-rich materials in permanently shadowed regions (PSRs). These resources are critical for "long-term human presence and resource sustainability" on the Moon.
- **Environmental Sustainability:** The computational implementation was designed to be efficient. By utilising Google Colab's cloud infrastructure and prioritising feature-based models like Random Forest over more energy-intensive deep learning architectures (unless resources allow), the project minimises the carbon footprint associated with local high-performance hardware.

3.4 Ethical Issues

The primary ethical considerations involve research integrity and the potential for algorithmic bias.

Data Integrity: There are no privacy or security risks associated with the lunar data used, as it contains no personal information. However, an ethical responsibility exists to report model performance honestly, including the "unsuccessful" cases where the ML model reduced robustness.

Algorithmic Bias: Machine learning models trained on specific lunar regions may struggle to generalise to other areas with different crater morphologies. Ethically, these limitations must be clearly stated to prevent the over-reliance on a model that might not perform as expected in unseen environments

AI-assisted development transparency:
A further ethical consideration concerns the transparent use of AI-assisted development tools and external coding resources during implementation. Tools such as ChatGPT, Claude, GitHub Copilot, online tutorials, and selected public repositories were used to support learning, code completion, restructuring, debugging, and implementation, where the author's prior technical knowledge was limited. However, the final architectural choices, adaptation of code to the Endymion pipeline, validation of outputs, benchmarking design, and interpretation of results remained the responsibility of the author. This distinction is important to maintain academic integrity: AI-assisted and external resources supported development, but they did not replace the author's ownership of the system design, integration decisions, and final evaluation.

3.5 Security Issues

Security in the context of Endymion relates to data and system integrity. As a navigation aid, the risk of "adversarial" input is low; however, ensuring the integrity of the hazard map and cost grid is paramount. Any corruption in the data flow between the DataHandler and the Pathfinder could result in a "ghost" safe path that actually leads into a crater or a steep slope. To mitigate this, the system incorporates automated sanity checks and validation layers to ensure that all inputs to the pathfinding algorithm are mathematically sound and free of anomalies.

4. Methodology

This chapter presents the methodology used in the development of Endymion. It describes the iterative lifecycle through which the system evolved, the broad phases and milestones that structured the project, and the testing strategy adopted to ensure correctness, reproducibility, and fair comparison across hazard-model variants. In doing so, it establishes the methodological basis for the final system design, implementation decisions, and evaluation results presented in subsequent chapters

4.1 Lifecycle approach

The development of Endymion followed an iterative, prototype-led lifecycle rather than a strictly linear model. This was appropriate because the project combined lunar data handling, terrain analysis, hazard modelling, pathfinding, and evaluation within one analytical pipeline, meaning that later stages could only be designed properly once earlier outputs had been validated in practice. The system therefore evolved through a series of working versions, where each stage exposed the limitations of the previous one and informed the next revision. This aligns with the final report's emphasis on modularity, reproducibility, and controlled comparison between hazard-model variants.

The first stage focused on a constrained terrain-only prototype. It aimed to prove that a south-polar DEM region could be loaded, converted into slope and roughness rasters, fused into a hazard surface, and then used to generate a valid route. This established the minimum working chain of the system, but it also revealed important issues. The early hazard surface could become too saturated, pathfinding searches could expand too much of the grid, and the notebook-centred workflow made repeatable comparison difficult. These issues showed that the project needed not only functional correctness, but also better interpretability, scalability, and reproducibility.

The next stage, therefore, shifted towards architectural consolidation. The project moved from a prototype workflow into a more structured modular pipeline, and the spatial contract was stabilised through a GeoTIFF-first workflow and a raster-defined canonical ROI. This made terrain products, crater products, hazard maps, navigation outputs, and evaluation artefacts directly comparable across runs. It also helped separate the official pipeline runner, the benchmark runner, and the later ML bridge, preventing new experiments from becoming disconnected extensions of the prototype.

A final stage introduced controlled extension. Crater-aware products were added first, followed later by a machine-learning bridge for crater probability. However, these additions were not allowed to alter the wider route-planning contract. Instead, the core planning settings were kept fixed so that later improvements could be compared fairly against the terrain-only baseline. In that sense, the Endymion lifecycle was not simply about adding more features, but about turning a working prototype into a reproducible and defensible analytical framework.

Some inconveniences during development caused the original planned dates to shift or be altered. The main causes were technical rather than organisational:

- The need to refactor away from the earlier prototype workflow,
- To stabilise the GeoTIFF-based pipeline,
- To formalise caching and saved artefacts,

- To ensure that crater-aware and ML-assisted extensions could be evaluated under fair benchmark conditions.

These changes made the development timeline less linear than first planned, but they also made the final methodology stronger, because each delay reflected a real improvement in experimental rigour rather than unstructured scope expansion.

4.2 Project phases and milestones

The project was organised into a number of broad phases that followed the Endymion schedule maintained in Notion.

Although that schedule was used more as a planning guide than as a strict historical log, it still provides a useful structure for explaining how the final methodology evolved.

The first phase was prototype development, running from late December 2025 to early February 2026. Its purpose was to produce a working terrain-based version of Endymion from data loading through to route generation. Within this phase, the DataHandler for south-polar LOLA data was scheduled between 2 and 7 January 2026, followed by the Feature Extraction module between 7 and 11 January, the Hazard Assessment model between 14 and 18 January, and the A* / Weighted A* pathfinding stage between 21 and 26 January. Evaluation and metrics followed at the end of January, after which benchmarking and scenario testing were introduced just before the first prototype integration and submission stage in early February.

Methodologically, this phase established the terrain-only analytical baseline. It confirmed that terrain-derived hazard modelling could support route generation and quantitative evaluation, but it also made clear that a prototype alone would not be enough for fair comparison or repeatable experimentation. As a result, a second phase emerged around refactoring and consolidation. This phase was less visible in the original plan, but became essential in practice because the project needed a cleaner pipeline structure, explicit artefact reuse, and a stable ROI contract before more advanced hazard variants could be evaluated properly.

The next main phase involved crater-aware and ML-assisted extension. In the schedule, crater ML integration with the hazard model was planned for mid-March 2026, followed by full system evaluation in late March. This phase differed from the original prototype work because it no longer aimed to prove that the pipeline worked at all, but rather to test whether richer hazard inputs could be integrated without breaking reproducibility or the fairness of benchmark comparison.

The final phase focused on documentation and consolidation. By late March and April 2026, attention shifted towards writing up the completed modules and explaining the methodology of the final implemented system. This phase matters because it reflects the transition from development to academic justification: the completed pipeline had to be documented not only as code that worked, but as a coherent methodological process.

Overall, the phases can be summarised as: early terrain-only prototype development, completion of hazard and navigation, introduction of benchmark scenarios, transition to a reproducible final pipeline, extension into crater-aware and ML-assisted models, and final report consolidation. Although some deadlines moved, the sequence remained consistent with the final system described in Chapters 5, 6, and 7.

4.3 Testing strategy

Testing formed a central part of the Endymion methodology rather than being treated as a final stage performed only after implementation. This was important because the project did not aim merely to produce hazard maps and routes, but to show that these outputs were valid, reproducible, and comparable under controlled conditions. The testing strategy was therefore organised around three levels: component verification, end-to-end pipeline validation, and controlled benchmark comparison. The specific tests later presented in Chapter 7 should be understood as practical executions of this strategy.

The first level was component verification. Each major module was checked against its expected contract before being relied upon by later stages. This included DEM loading, ROI resolution, feature extraction, crater-raster generation, hazard construction, cost conversion, pathfinding, and evaluation. At this stage, testing focused on output shape, valid ranges, spatial alignment, and safe failure handling. Examples included invalid data paths, incorrect ROI sizes, missing crater inputs, out-of-bounds coordinates, and cases where no valid route should exist. This level mattered because Endymion is a modular raster-based framework: if one module produced invalid or misaligned outputs, later hazard and navigation results would no longer be trustworthy.

The second level was end-to-end pipeline validation. Here, the goal was to confirm that the official pipeline runner could execute the full terrain-to-route chain under reproducible conditions. A successful run therefore meant more than just avoiding crashes. It meant resolving the correct ROI, reusing or rebuilding the expected cached artefacts, generating valid hazard and cost rasters, returning either a route or a controlled failure state, and saving outputs in a consistent structure for later inspection. This is why the final implementation makes such strong use of saved artefacts and structured metadata: these are not only practical outputs, but part of the validation method itself.

The third and most important level was controlled benchmark comparison. This level is the one that most directly supports Chapter 7. The strategy was to hold constant the canonical ROI, the selected benchmark cases, the cost conversion, the pathfinding settings, and the evaluator schema, while allowing only the hazard model to vary. This made it possible to compare terrain-only, crater-aware, and ML-assisted variants under the same route-planning conditions. As a result, differences in success rate, path hazard, cost per metre, or expansions could be interpreted as genuine effects of the hazard model rather than as side effects of planner reconfiguration. In other words, the benchmark tests later shown in the report were not ad hoc experiments, but the direct implementation of a prior comparison strategy.

A final part of the testing strategy was reproducibility. Repeated runs under the same settings were expected to resolve the same ROI, preserve the same artefact structure, and produce stable summaries. Cache reuse also formed part of this logic, because the system needed to demonstrate that derived terrain and crater products could be reloaded consistently instead of recomputed in inconsistent ways. Visual checks, such as hazard and route overlays, were also used to confirm spatial plausibility, but these were treated as supporting evidence rather than the primary basis of validation.

Taken together, this strategy gave the project a clear testing logic:

First verify each module locally, then verify the complete pipeline, and finally compare hazard variants under a fixed benchmark contract. Presenting the tests in Chapter 7 after this

methodological explanation, therefore makes the later results appear as the execution of a defined strategy, which is closer to the final structure and purpose of the completed Endymion system.

5. Design

This chapter describes the final design of Endymion as a modular lunar hazard-analysis and navigation framework. While the previous chapters established the project context, objectives, and methodological approach, this chapter focuses on how the implemented system was structured to transform south-polar terrain data into hazard-aware navigation outputs. The design emphasises reproducibility, modular responsibility, explainable hazard fusion, and controlled comparison between hazard-model variants. It therefore presents the architectural structure, data flow, hazard model, pathfinding logic, and diagrammatic representation used to support the final implemented pipeline.

5.1 System architecture

The final Endymion architecture was designed as a modular hazard-navigation pipeline rather than as a single-purpose prediction model. Its structure separates data ingestion, terrain feature generation, crater-related processing, hazard fusion, route planning, and evaluation into distinct but connected components.

The execution flow is:

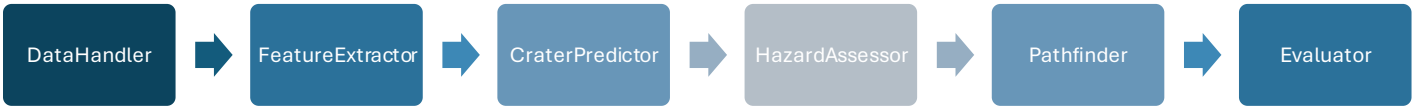


Figure 1 - System's flow

This design supports reproducibility, clearer traceability of intermediate artefacts, and controlled comparison between hazard-model variants. While the system includes a machine-learning extension for crater-probability generation, the architecture as a whole remains primarily data-driven and analytical, with machine learning acting as one component within a broader hazard-assessment and navigation framework.

At the architectural level, the system is centred on a canonical south-polar Region of Interest (ROI) extracted from the GeoTIFF DEM workflow. The DataHandler resolves the raster source, determines the canonical ROI directly from the raster grid, and supplies a consistent processing patch for all later stages. In the final workflow, this centred ROI acts as the common spatial reference for terrain features, crater rasters, hazard maps, route generation, and saved evaluation artefacts. This is important because it removes dependence on earlier prototype indexing choices and ensures that all later experiments operate on the same spatial contract.

A second architectural feature is the separation between the official pipeline, the benchmark runner, and the ML bridge. The main pipeline is responsible for one reproducible end-to-end run, the benchmark runner repeats comparable runs across fixed cases and hazard variants, and the

crater ML bridge produces a probability raster that can later be consumed as an additional hazard input. This keeps the core system stable while still allowing controlled methodological extension.

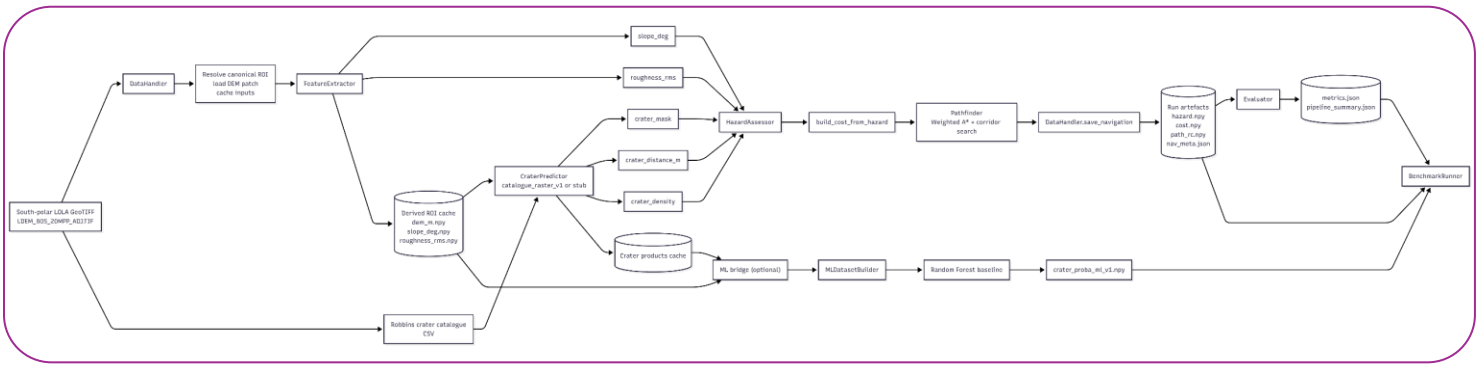


Figure 2 - Final Endymion pipeline architecture showing the core modules, saved artefacts, and extension paths for benchmarking and ML-assisted processing.

5.2 Data flow and module responsibilities

The data flow begins with the DEM source and a fixed ROI definition. First, the *DataHandler* loads the south-polar *GeoTIFF*, resolves the canonical ROI, extracts the DEM patch, and manages persistent and runtime caching. The FeatureExtractor then derives aligned terrain rasters, principally slope and RMS roughness, from the DEM patch. These products are saved as reusable derived artefacts, allowing later executions to reload them instead of recomputing them. In the implemented pipeline, this derived stage produces artefacts such as dem_m.npy, slope_deg.npy, and roughness_rms.npy.

If crater processing is enabled, the *CraterPredictor* either loads cached crater products or builds them from the Robbins catalogue through an ROI-aligned rasterisation workflow. In practice, this produces a crater mask, crater-distance, and crater-density rasters in the same grid as the DEM-derived features. Because all products remain ROI-aligned, the hazard stage can consume them directly without additional resampling or coordinate reconciliation.

The HazardAssessor then fuses the terrain and crater signals into a single hazard raster. That hazard raster is subsequently converted into a traversal-cost raster and passed to the Pathfinder together with ROI-local start and goal coordinates. The Pathfinder then computes a route under the defined safety and search constraints, after which the DataHandler saves the navigation outputs. Finally, the Evaluator reads the saved run artefacts and produces quantitative metrics for path geometry, safety, and efficiency. In the final pipeline, the saved navigation contract includes hazard.npy, cost.npy, path_rc.npy, and nav_meta.json, while evaluation adds metrics.json and a pipeline summary

The responsibilities of each module are therefore clearly bounded. The *DataHandler* manages acquisition, validation, caching, and ROI extraction; the *FeatureExtractor* derives terrain descriptors; the *CraterPredictor* exposes crater-aware spatial products; the *HazardAssessor* converts multiple hazard cues into a unified risk surface; the *Pathfinder* optimises route selection under that surface; and the *Evaluator* measures the quality of the resulting route.

This modular split is one of the main design strengths of Endymion because it supports debugging, reproducibility, and future additions without forcing a redesign of the full pipeline.

The main saved artefacts are as follows:

derived terrain artefacts - *(dem_m.npy, slope_deg.npy, roughness_rms.npy, meta.json);*

crater artefacts - *(crater_mask.npy, crater_mask_distance.npy, crater_mask_density.npy, crater_mask_meta.json);*

navigation artefacts - (*hazard.npy, cost.npy, path_rc.npy, nav_meta.json);*

evaluation artefacts - (*metrics.json, pipeline_summary.json*);

ML artefacts - (*ml_dataset_rf_v1.csv, crater_proba_ml_v1.npy, model and summary files*);

and benchmark summaries - (*manifest.json, summary.json, summary.csv*).

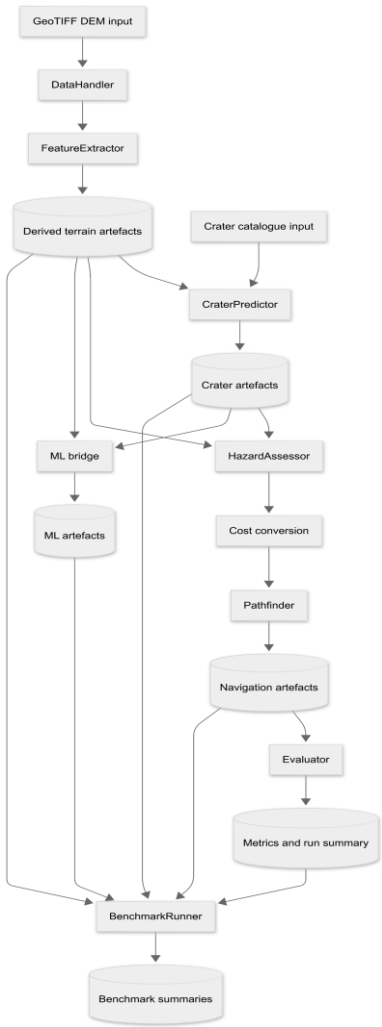


Figure 3 - Data flow diagram showing the saved artefacts exchanged between modules in the final pipeline.

5.3 Hazard model design

The hazard model was designed as an explainable fusion layer rather than as a black-box predictor. In its final implemented form, the HazardAssessor supports both a terrain-only mode and a terrain-plus-crater mode. The terrain component is based on normalised slope and normalised roughness, combined by a weighted sum.

When crater inputs are present, a second component is added using the crater mask, crater-distance risk, and crater density. The final hazard is then formed by combining the terrain and crater components through group-level weights.

This can be expressed as:

$$\boldsymbol{H}_{\text{terrain}} = \boldsymbol{w}_s\widehat{\boldsymbol{S}} + \boldsymbol{w}_r\widehat{\boldsymbol{R}}$$

$$\boldsymbol{H}_{\text{crater}} = \boldsymbol{w}_m\widehat{\boldsymbol{P}} + \boldsymbol{w}_d\widehat{\boldsymbol{D}} + \boldsymbol{w}_c\widehat{\boldsymbol{C}}$$

$$\boldsymbol{H} = \boldsymbol{w}_t\boldsymbol{H}_{\text{terrain}} + \boldsymbol{w}_k\boldsymbol{H}_{\text{crater}}$$

Where:

$\hat{S}$ is the normalised Slope term

$\hat{R}$ is the normalised Roughness term,

$\hat{P}$ represents the Crater-Presence

$\hat{D}$ represents the Crater-Distance risk

$\hat{C}$ represents the Local-Crater Density

This formulation was kept deliberately simple so that the contribution of each factor remains interpretable during testing and benchmarking.

A further design decision was the use of an explicit impassable-slope rule. After fusion, cells above the defined slope threshold are forced to maximum hazard before the surface is clipped to the [0,1] interval. This gives the model a clear safety boundary and prevents the final route from treating extreme terrain as merely expensive rather than effectively non-traversable. In design terms, this is important because Endymion is not only predicting relative risk but also enforcing a minimum safety policy before planning begins.

The model was also designed to remain extensible. Alongside crater-aware fusion, a separate ML bridge was introduced to generate an ROI-aligned crater-probability raster from cached terrain products. Although this probability raster is not required for the core pipeline, the design allows it to be incorporated later as an additional hazard signal within the same benchmarking framework. This preserves architectural stability while allowing later refinement of the hazard layer without changing the wider navigation contract.

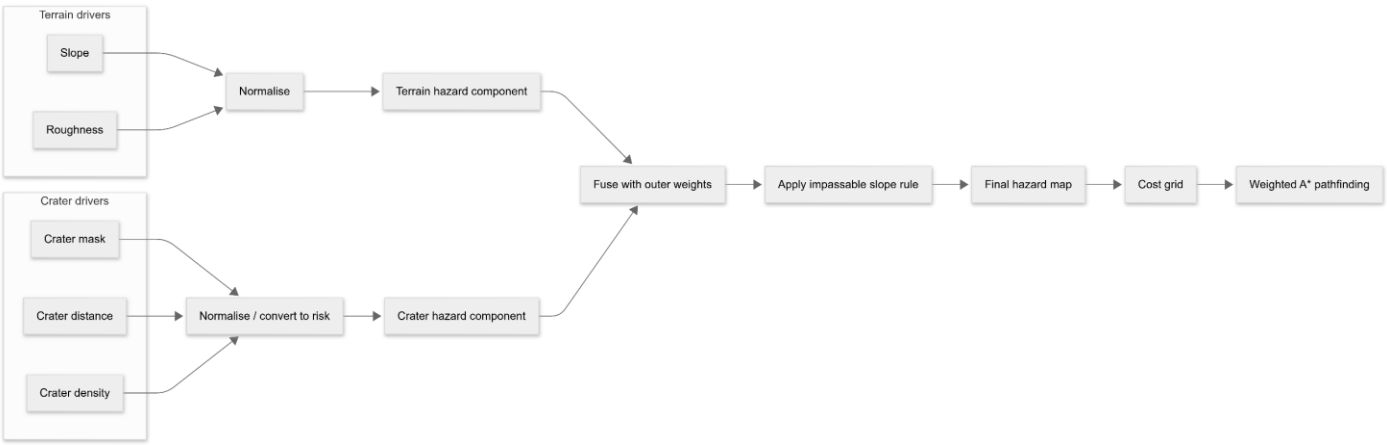


Figure 4 - Hazard fusion logic in the final Endymion hazard model

5.4 Pathfinding design

Pathfinding in Endymion is performed over a traversal-cost raster derived directly from the hazard map.

The implemented cost transformation is linear:

$$\boldsymbol{Cost = 1 + \alpha H}$$

Cells at or above the hazard blocking threshold are assigned a very large blocking cost. This design preserves two requirements simultaneously: distance still matters through the base cost term, while hazardous terrain is penalised strongly enough to alter route selection. The explicit blocking rule also ensures that safety-critical regions are excluded before the search begins.

The planner itself is a Weighted A* implementation operating on a two-dimensional grid with configurable 4- or 8-neighbour connectivity. Its evaluation follows the following standard weighted form:

$$\mathbf{f}(\mathbf{n}) = \mathbf{g}(\mathbf{n}) + \mathbf{w}\,\mathbf{h}(\mathbf{n})$$

Where:

$g(n)$ is the accumulated path cost

$h(n)$ is the heuristic estimate to the goal

w is the heuristic weight

In the implemented version, increasing the heuristic weight is treated as an efficiency choice rather than a safety choice, because safety is already enforced by the blocked-cost mask. This distinction is useful academically: the hazard model defines what is safe, while the planner defines how efficiently the search reaches a safe path.

To improve tractability, the final design also includes corridor-restricted search. A corridor mask is constructed around the straight-line segment between start and goal, and the planner can search progressively wider corridor radii until a route is found. This reduces unnecessary exploration in large grids while retaining the same hazard and blocking logic. The result is a pathfinding design that is still conceptually simple, but more scalable than the earliest prototype, which tended to expand too much of the grid.

Finally, the pathfinding stage was integrated into a fixed comparison contract for benchmarking. In the final project structure, pathfinding settings such as connectivity, heuristic weight, corridor radii, and cost-conversion parameters are held constant while only the hazard model varies across benchmark cases. This is an important design point because it makes route differences interpretable as changes in hazard modelling rather than as hidden planner reconfiguration.

A short implementation extract of *build_cost_from_hazard*(...) and corridor-aware path execution is provided in the appendix rather than in the main design chapter.

5.5 UML & DFD Discussion

The UML and DFD material for Endymion was revised so that it reflects the final implemented system rather than the earlier proposal-stage concept of a monolithic controller. For this project, the UML diagram is most useful for showing the static responsibilities and dependencies between the principal modules: DataHandler, FeatureExtractor, CraterPredictor, HazardAssessor, Pathfinder, Evaluator, BenchmarkRunner, and the ML bridge. This helps demonstrate modularity, separation of concerns, and extension points. However, the diagram should avoid overstating elements that are not central to the final submission, such as a persistent operational database or

a large interactive control layer, because the implemented workflow is primarily file-based and pipeline-driven.

By contrast, the DFD is better suited to representing how data products are transformed across stages. In Endymion, the most meaningful DFD begins with the GeoTIFF DEM and crater catalogue, passes through ROI extraction and raster derivation, and then shows hazard generation, cost conversion, path computation, and evaluation outputs. It emphasises saved intermediate artefacts because these artefacts are central to reproducibility and later benchmarking.

The DFD therefore makes explicit that Endymion consumes analysis-ready spatial products and produces reusable outputs rather than streaming raw mission telemetry through a single opaque process.

The UML and DFD, therefore, serve complementary purposes. The UML diagram explains what each module is responsible for and how modules depend on one another, whereas the DFD explains how data moves and changes form across the pipeline. Used together, they show that Endymion is not merely a sequence of scripts, but a structured hazard-navigation framework with clear architectural boundaries, reproducible artefact flow, and controlled extension points for crater-aware and ML-assisted variants.

The runtime artefact flow discussed here is shown earlier in Section 5.2, while Figure 5 presents the final UML view of the implemented module structure.

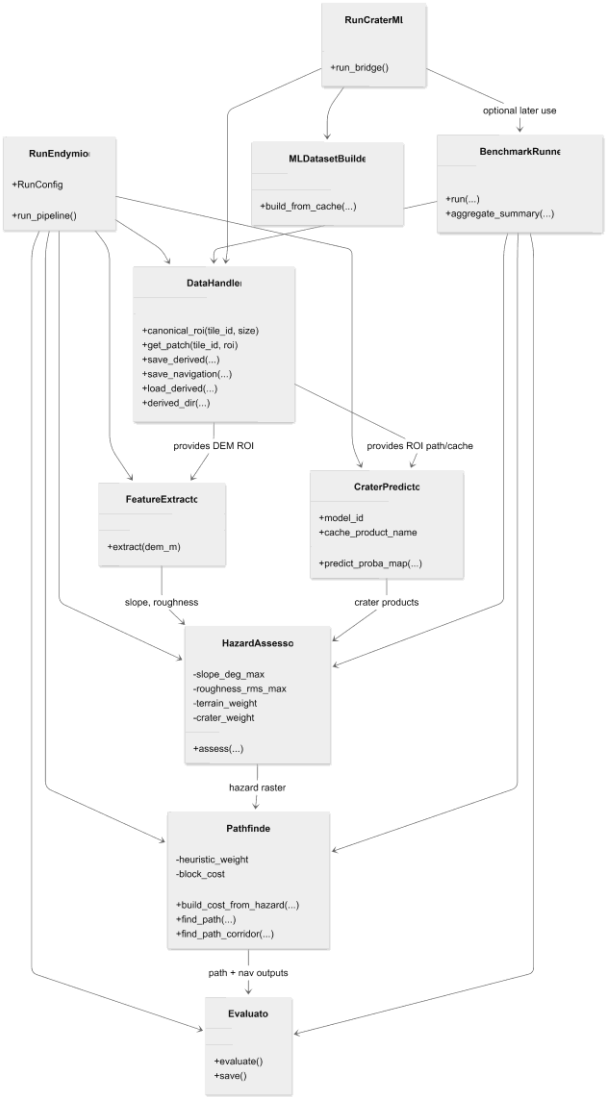


Figure 5 - Final UML-style component diagram of the implemented Endymion module structure.

6. Tools and Implementation

This chapter outlines how the design described in the previous chapter was translated into a functional software system. While Chapter 5 presented the conceptual architecture, data flow, and core algorithmic logic, this chapter focuses on the practical technologies, development decisions, and implementation steps used to build Endymion. It is divided into a review of the main tools used and a detailed explanation of how the final pipeline was implemented.

6.1 Tools

The Endymion system was developed primarily in Python due to its strong support for scientific computing, geospatial processing, machine learning, and rapid prototyping. Python was particularly suitable for this project because the pipeline required integration between the rasterisation handling, numerical terrain analysis, crater featuring generation, hazard modelling and evaluation.

Development was carried out across both notebook-based and local environments, with additional supporting tools used for version control, planning, and AI-assisted development.

- **Development environment:**

  Google Colab and *Visual Studio Code* were used during different stages of the project. Colab supported experimentation, notebook validation, and repeatable runs, while Visual Studio Code was used for local development, debugging, refactoring, and organising the final modular pipeline.

- **Core language and standard library:**

  Python formed the basis of the implementation. Standard modules such as *argparse, json, pathlib, typing, types*, and *importlib* were used for command-line execution, configuration handling, file management, modular imports, and clearer code structure.

- **Numerical and data processing libraries:**

  NumPy and *Pandas* were used throughout the project for array-based raster operations, feature processing, metadata handling, and machine learning dataset preparation.

- **Geospatial and machine learning libraries:**

  Rasterio was used to work with *GeoTIFF* data and aligned raster products, while scikit-learn was used for the Random Forest crater baseline, metric calculation, and dataset preparation. Matplotlib was used for visual validation and result plotting during notebook-based analysis.

- **Version control and project organisation:**

  GitHub was used to manage repository changes and support the transition from the early prototype to the cleaned final pipeline. Notion was used to organise notes, track progress, and document decisions throughout development.

- **AI-assisted support tools:**

  AI-based tools and external learning resources were used during development to support code completion, debugging, restructuring, implementation guidance, and technical understanding. These included ChatGPT, Claude, GitHub Copilot, online video tutorials, and selected public GitHub repositories used for learning and comparison. Their role was to assist development where prior implementation knowledge was limited, particularly during refactoring, module integration, and debugging. However, the final architectural decisions, integration of modules, adaptation of code to the Endymion pipeline, validation of outputs, benchmarking design, and interpretation of results remained the responsibility of the author

Together, these tools supported both the experimental and final stages of the project, from early prototyping to the construction of the final reproducible Endymion pipeline.

6.2 Implementation

The implementation of Endymion evolved across several stages, beginning with an early prototype centred on terrain-only hazard modelling and later on, developing into a modular pipeline that incorporated crater-aware analysis, reproducible benchmarking, and a separate machine-learning bridge for crater probability generation.

Instead of being developed as a single, fixed system from the beginning, the project evolved through iterative refinement, with each phase addressing specific technical limitations identified in the prior version.

The following subsections therefore describe the implementation in the same order in which the final system matured: from the initial prototype, through the architectural transition, to the final reproducible pipeline used for evaluation.

6.2.1 Early prototype development:

The early prototype of Endymion was developed as a deliberately constrained first implementation whose main purpose was to prove that the project could transform lunar elevation data into a usable navigation output. Rather than attempting full hazard fusion from the outset, the prototype focused on a smaller terrain-only pipeline: load a bounded LOLA south polar DEM region, extract terrain features, convert those features into a hazard surface, and generate a route across that surface. This narrow scope was intentional, as it reduced early system complexity while establishing a baseline that could later support crater-aware and machine-learning-based extensions

Figure 6 - Prototype pipeline overview (DEM → FeatureExtractor → HazardAssessor → Cost Grid → Pathfinder)

At this stage, the prototype relied on a single south polar DEM workflow and a fixed canonical Region Of Interest (ROI) so that each experiment would remain reproducible and directly comparable. The *DataHandler* was then responsible for resolving the raster source, loading the DEMs patch and keeping the outputs tied to a stable region of interest; meanwhile, the *Feature Extractor* generated the first terrain descriptors used by the system: Slope and roughness. Slope was derived from the DEM gradients, and roughness was estimated through local *RMS (root mean square)* surface variation, giving the prototype two interpretable indicators of traversal difficulty that could be reused throughout the rest of the pipeline. This was an important design decision because it ensured that the first prototype remained numerically simple, traceable, and also easy to validate before more complex hazard factors were introduced.

The first HazardAssessor was then intentionally terrain-only and used on a weighted hazard combination so that risks remain easy to interpret.

$$\mathbf{H} = \boldsymbol{w_s}\widehat{\boldsymbol{S}} + \boldsymbol{w_r}\widehat{\boldsymbol{R}}$$

Where $\hat{S}$ is the normalised Slope term

$\hat{R}$ is the normalised roughness term

w_s and w_r define their relative contribution

This means that steeper and rougher regions produce higher hazard values, while flatter and smoother areas remain more traversable. Although simple, this first formulation exposes an important issue during testing:

The Hazard surface initially became too saturated with too many cells, pushed close to the upper risk range. So, in order to correct this, the prototype normalisation was refined using more robust thresholding and an *impassable-slope* rule. Improving contrast without losing interpretability. This matter, as it was the surface that would directly control the path planning.

To make the terrain hazard usable for navigation purposes, the system then converted the hazard traversal into the traversal cost using a simple linear relation:

$$\boldsymbol{cost} = \mathbf{1} + \boldsymbol{\alpha H}$$

Where:

H is the Hazard value

α controls how strongly hazardous terrain penalises the movement

Cells above the chosen threshold were assigned to a very large blocking cost, effectively preventing the planner from crossing terrain considered unsafe. Route generation was then performed using the *A** algorithm, which was later configured as a *Weighted A** to improve the search efficiency.

Its evaluation rule is as follows:

$$\boldsymbol{f(n)} = \boldsymbol{g(n)} + \boldsymbol{w\,h(n)}$$

Where:

$g(n)$ is the accumulated path cost

$h(n)$ is the heuristic estimate to the goal

w is the heuristic weight

In practice, this allowed the prototype to produce valid hazard-aware routes while also revealing an important limitation: although the planner worked, early searches expanded too many nodes, showing that correctness had been achieved before efficiency.

That finding was valuable because it confirmed that the prototype had already met its real objective: establishing a working terrain-to-route baseline and exposing the main issues that would justify the later architectural transition.

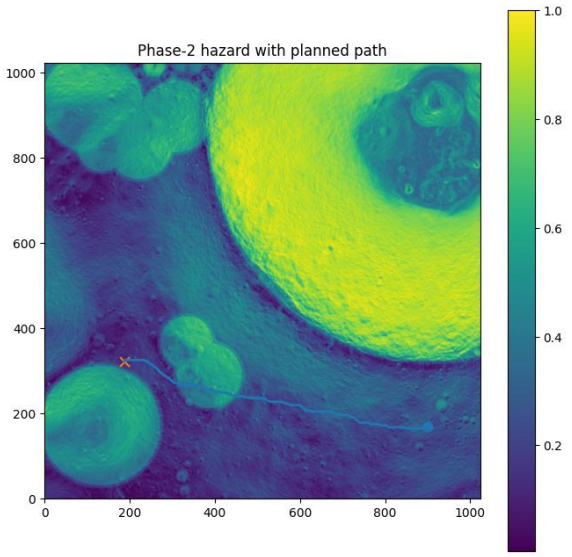


Figure 7 - Path overlay on DEM showing the terrain-only prototype result

Overall, the early prototype should be understood not as a reduced version of the final system, but as the first stable proof of concept. It established the project's core computational chain, validated the use of terrain-derived hazard modelling, and identified the main technical shortcomings that had to be addressed in the next stage, particularly hazard expressiveness, crater integration, and search scalability

6.2.2 Transition to the final pipeline architecture

Once the early prototype had shown that DEM-derived terrain features could generate both a usable hazard surface and a valid route, the main issue was no longer correctness but architectural stability. Although the terrain-only chain worked, it also exposed the weakness of a notebook-centred workflow: data loading, feature extraction, hazard modelling, and navigation were still too tightly coupled for repeatable experimentation or fair comparison between later hazard variants. For that reason, the next stage of development focused on refactoring Endymion into a cleaner and more reproducible pipeline.

A major part of this transition was the move towards analysis-ready products rather than raw mission telemetry. Instead of continuing to rely on the earlier IMG/LBL-centred prototype workflow, the final pipeline adopted a GeoTIFF-first DEM workflow, using the south-polar LOLA product as the main terrain backbone. At the same time, the canonical ROI was redefined directly from the raster grid rather than inherited from earlier prototype coordinates. This provided a stable spatial reference for feature extraction, crater rasterisation, hazard generation, navigation, and later benchmarking.

The second major change was the clearer separation of responsibilities across modules. In the refactored architecture, the DataHandler managed DEM access and caching, the FeatureExtractor generated aligned terrain rasters, the CraterPredictor built or loaded crater products, the HazardAssessor fused hazard terms, the Pathfinder optimised traversal across the cost grid, and the Evaluator measured safety and efficiency after each run. This modular structure meant that the system was no longer treated as a single notebook experiment, but as a reusable sequence of components connected through saved artefacts.

The final run_endymion.py script reflects this directly through the pipeline flow:

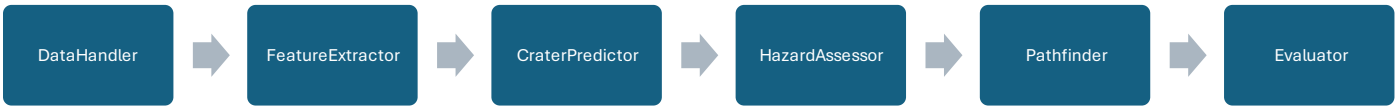


Figure 8 - Endymion's pipeline flow

A further improvement was the introduction of a clearer reproducibility contract. Terrain products such as *dem_m.npy, slope_deg.npy,* and *roughness_rms.npy* were saved as derived artefacts, while each navigation run stored outputs such as *hazard.npy, cost.npy, path_rc.npy*, and *nav_meta.json*, which were then consumed by the evaluator to produce *metrics.json*. This made each run traceable and reusable, while also separating the official pipeline runner, the benchmark runner, and the presentation notebook.

The refactor also made later extensions possible without breaking the terrain workflow. Crater catalogue data could now be converted into ROI-aligned raster products before entering hazard fusion, allowing crater-aware hazard modelling to remain compatible with the main pipeline. Likewise, the machine-learning bridge could be added as a separate stage that builds a dataset from cached ROI rasters, trains a baseline Random Forest model, and saves crater-probability outputs for later benchmarking. In this way, both crater and ML stages became controlled extensions of the same core system rather than isolated experiments.

Phase-1 vs Phase-2 comparison

```
print("=== Terrain-only ===")
print("Planner total cost:", metrics_base["efficiency"]["planner_total_cost"])
print("Cost sum along path:", metrics_base["efficiency"]["cost_sum_along_path"])
print("Path length (m):", metrics_base["geometry"]["path_length_m"])
print("Mean hazard:", metrics_base["safety"]["path_hazard_mean"])
print("Max hazard:", metrics_base["safety"]["path_hazard_max"])
print("Safety score:", metrics_base["safety"]["safety_score"])
print("Expansions:", metrics_base["efficiency"]["expansions"])
print("Cost per meter:", metrics_base["efficiency"]["cost_per_m"])

print("\n=== Terrain + crater ===")
print("Planner total cost:", metrics["efficiency"]["planner_total_cost"])
print("Cost sum along path:", metrics["efficiency"]["cost_sum_along_path"])
print("Path length (m):", metrics["geometry"]["path_length_m"])
print("Mean hazard:", metrics["safety"]["path_hazard_mean"])
print("Max hazard:", metrics["safety"]["path_hazard_max"])
print("Safety score:", metrics["safety"]["safety_score"])
print("Expansions:", metrics["efficiency"]["expansions"])
print("Cost per meter:", metrics["efficiency"]["cost_per_m"])
```

```
=== Terrain-only ===
Planner total cost: 2715.77978515625
Cost sum along path: 2652.6055183410645
Path length (m): 15704.894654563448
Mean hazard: 0.27099377943294034
Max hazard: 0.547321617603302
Safety score: 0.987812770968769
Expansions: 36955
Cost per meter: 0.1689031080237322

=== Terrain + crater ===
Planner total cost: 2715.77978515625
Cost sum along path: 2494.3291190862656
Path length (m): 15664.894654563448
Mean hazard: 0.24983578180269891
Max hazard: 0.663292646408081
Safety score: 0.9887563843670581
Expansions: 36955
Cost per meter: 0.15923050707268085
```

Finally, the transition introduced a fixed comparison contract for Phase 2. Instead of changing the cost model, pathfinding parameters, and evaluation schema alongside the hazard model, those elements were kept fixed and only the hazard-generation stage was allowed to vary. This made fair comparison possible between terrain-only, crater-aware, and ML-assisted hazard configurations, and turned the benchmark runner into an architectural component rather than a simple testing script.

Move it to the appendix – not implementation but evaluation

Figure 9 - Phase 1 vs Phase 2 benchmark comparison

Overall, this transition marked the shift from a working prototype to a repeatable experimental framework. By introducing a GeoTIFF-first workflow, a stable canonical ROI, modular responsibilities, persistent run artefacts, and a fixed benchmarking contract, Endymion became suitable for controlled extension and fair evaluation in its later stages.

6.2.3 Crater-aware hazard modelling and pathfinding integration

The next stage of Endymion extended the terrain-only prototype into a crater – aware hazard model and connected the model to the navigation stage under the same pipeline. What was built

at this stage was not a completely new planner, but an extended hazard representation. In addition to the terrain terms already derived from the DEM, the system incorporated crater-aware raster products generated from a catalogue workflow:

- A binary crater mask,
- A distance-to-crater raster
- A local crater-density raster.

These products were aligned to the same canonical ROI as the DEM and then fused inside the "Phase 2" HazardAssessor, which allows crater information to influence risk without breaking compatibility with the earlier terrain-only implementation.

The crater-aware hazard model remained intentionally simple and explainable. The terrain component continued to use the weighted terrain sum introduced earlier, while the crater contribution was expressed through three interpretable terms:

- Crater presence
- Crater proximity
- Crater density

In simplified form, the fused hazard followed:

$$\boldsymbol{H_{terrain} = w_s\widehat{S} + w_r\widehat{R}}$$

$$\boldsymbol{H_{crater} = w_m\widehat{M} + w_d\widehat{D} + w_c\widehat{C}}$$

$$\boldsymbol{H = w_t H_{terrain} + w_k H_{crater}}$$

Where $\hat{S}$ and $\hat{R}$ are the normalised Slope and Roughness terms

$\widehat{M}$ is the Crater Mask term

$\widehat{D}$ is the Crater-Distance risk term

$\hat{C}$ is the Local Crater Density term

This structure was useful because it kept the model transparent; The slope and roughness still capture basic traversability, while the crater terms adds a geological context that the terrain-only prototype could not represent

To make the fuse hazard usable for navigation, the pathfinding kept the same cost transformation

$$\boldsymbol{cost = 1 + \alpha H}$$

With cells above the blocking threshold treated as effectively impassable.

The route generation uses Weighed A* with an evaluation based on:

$$\boldsymbol{f(n) = g(n) + w\,h(n)}$$

Where $g(n)$is the accumulated path cost, $h(n)$is the heuristic estimate to the goal, and $w$is the heuristic weight. In the later implementation, this was combined with a **corridor-restricted**

search, where the planner searched within progressively wider corridor masks around the straight-line connection between start and goal.

This was an important refinement because it improved tractability without changing the underlying safety logic: safety remained enforced by the hazard blocking mask, while Weighted A* and corridor restriction mainly reduced unnecessary search effort

The main problem that appeared at this stage was that crater-awareness became visible in the hazard and cost values, but did not always produce a clearly different route. In the first direct terrain-only versus terrain-plus-crater comparison, the planner returned effectively the same path geometry and the same number of expansions, even though the crater-aware model changed the mean hazard and cost measured along that route. In other words, the crater term was active, but not yet strong enough to alter the selected corridor under that specific weighting scheme. A broader multi-case benchmark then exposed a second issue: the crater-aware model was operational, but under its early fusion settings, it did not consistently outperform the terrain-only baseline and even failed one case in the initial benchmark set.

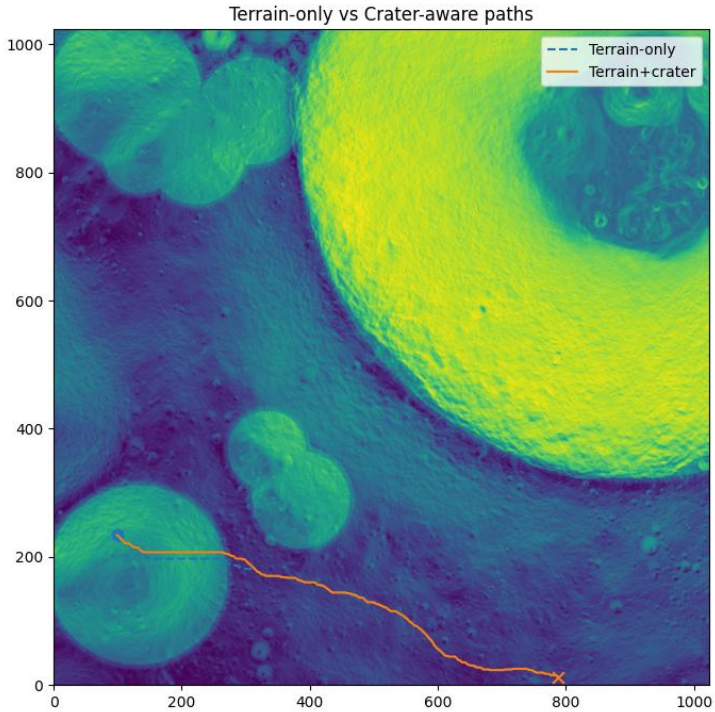
This was resolved not through a complete redesign, but through tighter experimental control. The pathfinding contract was frozen so that the cost model, hazard blocking rule, evaluator schema, and pathfinder settings remained fixed while only the hazard model varied. A multi-case, multi-model benchmark structure was then introduced so that terrain-only and crater-aware variants could be compared on the same cases, with the same start-goal pairs and the same planner configuration. This turned the problem from an unclear visual judgement into a measurable comparison of success rate, mean hazard, cost per metre, and expansions. It also made it possible to tune crater influence more cautiously, instead of changing multiple subsystems at once

The final outcome of this stage was therefore twofold. First, Endymion achieved a fully operational crater- aware hazard and pathfinding pipeline, in which the crater-derived products could be fused into a navigation without disrupting the earlier terrain workflow.

Second, the evaluation showed that the crater-awareness was genuinely present in the system, but, that its early weighting did not yet guarantee a better safety-efficiency trade-off than the terrain – only baseline.

That is still a valid outcome:

- Demonstrate successful integration
- Exposed the sensitivity of hazard fusion
- Created the controller benchmark framework required for the ML-assist stage



(Start: (234, 99)

Goal: (12, 788))

Figure 9 - Crater-aware hazard map and route overlay compared with the terrain-only baseline

6.2.4 Machine learning crater model and reproducibility bridge

The machine-learning stage in Endymion was implemented as a **baseline extension** rather than as the central driver of the system. Its purpose was not to replace the wider hazard-navigation pipeline, but to introduce a supervised crater-related signal that could be generated, saved, and evaluated within the same reproducible ROI-based framework as the other components. For this reason, the ML bridge was designed to consume cached terrain-derived rasters, construct a per-pixel supervised dataset, train a Random Forest classifier, and save an ROI-aligned crater-probability output for later use by downstream hazard models.

This design decision was important for two reasons. First, it allowed machine learning to be incorporated without disrupting the architecture already established by the terrain-only and crater-aware pipeline stages. Second, it made it possible to compare terrain-only, crater-aware, and ML-assisted hazard variants under the same evaluation and benchmarking contract. In this way, the Random Forest stage should be understood as an ML-assisted extension to the Endymion framework rather than as evidence that the whole system is fully AI-driven.

The first model was implemented as a *Random Forest Classifier* trained on terrain-derived predictors only. The crater mask was used strictly as the target label rather than as an input feature, and crater-distance or crater-density rasters were also excluded from the predictors in the first version to avoid “leakage” from already excluded from already crater- derived information.

In simple terms, the model produced a crater probability for each pixel by aggregating the outputs of multiple trees using the formula below:

$$\hat{\boldsymbol{\rho}}(\boldsymbol{x}) = \frac{1}{\mathrm{T}}\sum_{\boldsymbol{t}=\boldsymbol{1}}^{\boldsymbol{T}} \boldsymbol{P_t}(\boldsymbol{x})$$

Where:

T is the number of trees

$P_t(x)$ is the class probability estimated by tree T

The resulting probability surface was then saved as a ROI-aligned crater *(crater_proba_ml_v1.npy)*, making it compatible with later hazard fusion and benchmarking. This kept the model modest in scope, but technically useful: the aim was not to claim a final crater detector, but to create a valid first ML signal that could be tested under the same navigation framework as the earlier terrain and crater-aware baselines.

The main problem that appeared at this stage was not simply predictive performance, but experimental validity. A pixel-wise crater model can look successful while still being misleading if neighbouring pixels from the same local terrain pattern are split carelessly between training and testing. In addition, a notebook-only implementation would have made it difficult to reproduce the exact dataset construction, feature set, and saved outputs later during benchmarking. This meant that the challenge was twofold: avoiding spatial leakage and ensuring that the ML stage remained consistent with the rest of the Endymion pipeline

This was resolved through two architectural choices. First, the dataset builder introduced a block-based spatial split, grouping pixels by *block_id* so that training and test sets were separated by spatial blocks rather than by arbitrary random pixels, making the held-out evaluation more defensible because the test data were less likely to be near-duplicates of the training terrain. Second, the ML stage was formalised through a dedicated bridge script rather than being left out as a notebook – only procedure.

The runner built the dataset from cached ROI rasters, the trained Random Forest baseline, and reported the held-out metrics and feature importances, as well as saved the probability map in a form that could be consumed later by the benchmark runner. In this way, the ML stage became reproducible both as a modelling step, and as a pipeline artefact

The ML workflow improved reproducibility by adhering to the existing storage contract used in Endymion. The terrain cache served as the source for input rasters, and the dataset builder saved the constructed training table for traceability. Final model outputs were written back to the derived ROI directory, creating a reproducibility bridge.

This integrated the ML model into the overall evaluation process, making it comparable to previous terrain analysis and crater-aware hazard variants.

The result was a first crater-probability model and a repeatable integration path for ML in Endymion. The project achieved a baseline supervised classifier, a spatially safer train-test split, saved metrics, and ROI-aligned probability outputs for future hazard fusion and benchmarking. This advancement connected predictive modelling to system-level evaluation while maintaining reproducibility, ultimately extending the earlier hazard framework.

Figure 10 - ML bridge flow

6.2.5 Final implementation state and deferred components

By the end of development, Endymion had reached a stable implementation state in which the main analytical pipeline could be executed through a single reproducible runner. In practical terms, the final system could load a GeoTIFF DEM, resolve the canonical ROI, generate terrain-derived rasters, optionally build crater products, fuse hazard terms, convert hazard into traversal cost, run corridor-aware Weighted A* pathfinding, and evaluate the resulting route through saved metrics. This meant that the project had moved beyond isolated notebook experiments and into a controlled pipeline with explicit inputs, outputs, and repeatable execution

Another relevant part of the final state was the benchmark framework. By this point, Endymion could compare multiple hazard configurations under the same execution contract, including the terrain-only baseline, the handcrafted crater-aware model, and the ML-assisted crater variant. This was significant because it allowed the system to be judged based on consistent measures rather than on isolated visual inspections, such as:

- Success rate
- Mean path hazard
- Cost per metre
- Expansions

The benchmark results also clarified the project's practical outcome; the terrain-only baseline remained the strongest model overall, while the crater-aware and the ML-enhanced variants proved that additional hazard information could be integrated successfully, even if they did not yet replace the terrain baseline as the safest final choice

```
----------------------------------------------------------------------
Endymion run complete
======================================================================
Run dir      : C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\terrain_only_smoke_test_v2
Derived dir  : C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712
ROI          : (14688, 15712, 14688, 15712)
Run name     : terrain_only_smoke_test_v2
Success      : True
Failure      : None
Total cost   : 636.4488525390625
Expansions   : 8807
----------------------------------------------------------------------
Path length m: 3510.7821048680194
Mean hazard  : 0.2659058178985311
Max hazard   : 0.4357956051826477
Safety score : 0.9883965566706623
Cost / m     : 0.16154634529362283
Metrics path : C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\terrain_only_smoke_test_v2\metrics.json
```

Figure 11 - Endymion CLI run summary

Not all planned ideas were implemented as final core components, and this is an important part of the project's conclusion. Illumination modelling was identified early as an important hazard factor, but it was deferred in order to keep the system focused on terrain, crater awareness, and reproducible benchmarking. Likewise, real-time visualisation, full multi-tile lunar coverage, rover-system integration, and live deployment remained outside the final implementation scope. These were not omitted by accident, but by design: the project prioritised a defensible analytical core over breadth, and the resulting modular architecture was intended to make such extensions possible later without requiring a redesign of the main pipeline

The same applies to further model development. The random Forest crater model provided a useful first ML bridge, but the final results suggested that alternative learners, such as a secondary gradient-boost model, may improve calibration or produce smoother probability maps in future work. In that sense, the final implementation should be understood as complete in architectural terms, but intentionally selective in methodological scope, as it delivered a functioning, reproducible, and benchmarkable lunar hazard-navigation framework, while leaving higher complexity for future work rather than forcing it prematurely into the submitted system

7. Testing and Results

This section details the testing strategies and results for the **Endymion** prototype, demonstrating how the system meets its technical requirements through structured verification, validation, and benchmarking.

7.1 Test Coverage

This section explains how the implemented system was tested against both the functional and non-functional requirements defined for the Endymion prototype. Functional coverage should focus on the core pipeline stages: DEM loading and validation, ROI extraction, terrain feature generation, hazard map creation, and safe path computation. Non-functional coverage should focus on reproducibility, modularity, ROI-based processing, validation checks, and support for later extensions such as ML and illumination modelling. A compact test case summary table can then be used to show the main tests performed, the expected behaviour, the observed result, and the final status.

7.1.1 Functional requirements coverage

Table 2 - Functional requirements

Test ID	Functional area	Key CLI setup	Expected result	Actual result (abridged)	Status	Evidence
T1	DEM loading and canonical ROI resolution	Local GeoTIFF, canonical ROI, no craters, no metrics	System loads the GeoTIFF, resolves the canonical ROI, and completes a valid terrain-only run	GeoTIFF loaded successfully; canonical ROI resolved to **(14688, 15712, 14688, 15712)**; run completed successfully with **Success=True** and **8807 expansions**	Pass	App. Fig. T1.1
T2	Invalid DEM path handling	Missing GeoTIFF path,	System rejects invalid DEM input with a clear file error	Execution failed safely with **FileNotFoundError: Local GeoTIFF not found**	Pass	App. Fig. T1.2

		no craters, no metrics				
T3	Manual ROI extraction	Explicit ROI (14688, 15712, 14688, 15712), no craters, no metrics	System accepts a valid manual ROI and completes the run	Manual ROI accepted exactly as provided; run completed successfully with **Success=True** and the same total cost/expansion pattern as the canonical case	Pass	App. Fig. T1.3
T4	Invalid ROI size rejection	Canonical ROI request with --roi-size 50000	System rejects an ROI request larger than the raster dimensions	Execution failed safely with **ValueError: Requested size 50000 exceeds raster shape (30400, 30400)**	Pass	App. Fig. T1.4
T5	Terrain feature generation	Canonical ROI, --rebuild-derived, no craters, no metrics	System rebuilds terrain-derived products and completes a valid run	Derived terrain stage completed successfully in **0.20 s**; run completed with **Success=True** and saved outputs to the navigation directory	Pass	App. Fig. T1.5
T6	Fresh crater-product generation and crater-aware execution	Fresh manual ROI, valid crater CSV, crater rebuild enabled	System builds crater products from the catalogue, fuses crater-aware hazard terms, and completes a valid run	Fresh crater-aware run completed successfully; ROI listing showed crater and terrain artefacts, and nav_meta.json confirmed **source: built_from_catalogue**; evaluator metrics were produced	Pass	App. Fig. T1.6
T7	Invalid crater catalogue handling	Fresh manual ROI, missing crater CSV path	System rejects missing crater-catalogue input with a clear file error	Execution failed safely with **FileNotFoundError: Robbins CSV not found**	Pass	App. Fig. T1.7
T8	Unified hazard generation	Canonical ROI, no craters, evaluator enabled	System generates a terrain-only hazard map, traversal cost, path output, and metrics	Terrain-only run completed successfully with **Success=True**, evaluator metrics saved, and hazard-aware path statistics reported	Pass	App. Fig. T1.8
T9	Valid hazard-aware pathfinding	Canonical ROI, no craters, valid ROI-local start/goal	System computes a valid route and saves navigation/evaluation outputs	Valid route returned with **Success=True**, **Total cost 636.45**, **Expansions 8807**, and evaluator metrics produced despite minor terminal formatting issues	Pass	App. Fig. T1.9
T10	Out-of-bounds coordinate rejection	Canonical ROI, invalid start (2000, 2000)	System rejects a start coordinate outside ROI-local bounds	Execution failed safely with a clear bounds-validation error explaining the valid ROI-local range	Pass	App. Fig. T1.10
T11	Safe failure under fully blocked conditions	Canonical ROI, no craters, --hazard-block 0.0	System should terminate safely without a route when all traversable cells are blocked	Run completed safely with **Success=False** and **Failure=start_or_goal_blocked**; no path metrics were produced beyond the saved failure state	Pass	App. Fig. T1.11
T12	ML bridge and crater-probability output	Canonical ROI, cached terrain/crater inputs, RF	System builds the ML dataset, trains the baseline classifier, evaluates it on a held-	ML bridge completed successfully; held-out metrics were accuracy 0.7941, precision 0.8942, recall 0.7291, F1 0.8033; probability	Pass	App. Fig. T1.12

		baseline via run_crater_ml	out split, and saves a crater-probability raster plus summary artefacts	raster, model, feature-importances file, and summary JSON were saved to the ROI derived directory		

The functional tests confirmed that Endymion operated correctly across both normal and failure conditions. Positive cases showed that the system could load and validate the lunar DEM, resolve both canonical and manual ROIs, rebuild terrain-derived products, generate crater-aware inputs from catalogue data, produce unified hazard outputs, compute valid navigation paths, and execute the ML bridge successfully. Negative cases also demonstrated that invalid DEM paths, oversized ROI requests, missing crater-catalogue inputs, out-of-bounds coordinates, and fully blocked navigation conditions were handled safely through explicit validation errors or controlled failure states.

Taken together, these tests provided evidence that the implemented pipeline satisfied the project's core functional requirements for data loading, feature extraction, hazard generation, and safe path computation

7.1.2 Non-functional requirements coverage

The non-functional tests focused on properties that affect the reliability and maintainability of the Endymion pipeline rather than its immediate computational outputs. In particular, testing considered reproducibility, cache reuse, safe validation behaviour, and extensibility of the modular architecture. These areas were important because the project was designed not only to produce hazard-aware navigation results, but also to do so in a deterministic, traceable, and incrementally extendable way

Test ID	Non-functional area	Evidence used	Actual result (abridged)	Status	Evidence
N1	Reproducibility	Repeated terrain-only execution under the same configuration	Repeated runs produced the same ROI, success state, total cost, and expansion count, indicating deterministic behaviour	Pass	App. Fig. N.1.2.1
N2	Cached artefact reuse	Rerun of existing ROI without forced rebuild	Cached DEM and ROI artefacts were reused successfully, and the derived terrain stage was completed quickly without unnecessary re-computation	Pass	App. Fig. N.1.2.2
N3	Validation and safe failure	Invalid ROI-local start coordinate	The system rejected invalid pathfinding input with a clear bounds-validation error rather than failing silently	Pass	App. Fig. N.1.2.3
N4	Extensibility/modularity	Existing crater-aware and ML bridge runs	Crater-aware and ML-assisted stages integrated successfully through the same ROI cache and saved artefact contract, without requiring redesign of the base runner	Pass	App. Fig. N.1.2.4

Table 3 - Non-Functional requirements

The non-functional results showed that Endymion behaved as a reproducible and modular analytical system rather than as a one-off notebook prototype. Repeated runs remained consistent, cached artefacts reduced unnecessary re - computation, invalid inputs were caught through explicit validation checks, and later crater-aware and ML-assisted stages could be incorporated through the same saved-artefact workflow. Together, these results supported the project's non-functional goals of reproducibility, modularity, controlled scale, and extensibility

7.1.3 Test case summary

Table 7.1.3 provides a compact summary of the main functional tests carried out on the Endymion pipeline. Unlike Section 7.1.1, which maps each test to requirement coverage in more detail, this subsection gives a concise overall view of what was tested, under what condition, and whether the implemented system behaved as expected. All twelve functional tests were ultimately passed. The only corrective case was T12, where the ML bridge was rerun successfully using python -m *src.run_crater_ml*, after which the crater-probability artefacts were generated correctly.

Test ID	Area tested	Input/condition	Expected result	Actual result	Pass / Fail
T1	DEM loading	GeoTIFF tile requested from cache or local source	DEM loads correctly	DEM loading completed successfully for the selected lunar tile	Pass
T2	Canonical ROI resolution	1024 × 1024 canonical ROI requested	Valid ROI returned	Canonical ROI was resolved correctly and reused consistently	Pass
T3	Feature extraction	DEM patch processed	Slope and roughness rasters generated	Derived terrain rasters were produced and saved successfully	Pass
T4	Crater raster generation	Robbins catalogue with DEM CRS and ROI	ROI-aligned crater products created	Crater mask, crater-distance, and crater-density products were generated and cached	Pass
T5	Terrain hazard generation	Slope and roughness inputs supplied	Hazard raster produced in valid range	Terrain hazard map was generated successfully	Pass
T6	Cost-grid construction	Hazard raster converted to traversal cost	Cost grid built with blocking threshold applied	Traversal-cost raster was produced correctly for planning	Pass
T7	Pathfinding	Valid ROI-local start and goal provided	Feasible path returned, or safe controlled failure	Weighted A* corridor search returned valid paths under the tested conditions	Pass
T8	Evaluation	Saved navigation artefacts available	Metrics file generated	Evaluator produced path, hazard, and efficiency metrics successfully	Pass
T9	End-to-end pipeline execution	Official pipeline runner executed	Full workflow completes and stores outputs	End-to-end run completed with saved outputs and summary artefacts	Pass
T10	Benchmark execution	Fixed cases and fixed settings used across models	Comparative benchmark outputs produced	Multi-case benchmark completed, and summary files were generated	Pass
T11	Crater-aware comparison	Terrain-only and terrain-plus-crater models tested on the same cases	Crater-aware variant executes and can be compared fairly	Comparative runs completed successfully, although the terrain-only baseline remained stronger overall	Pass
T12	ML crater-probability generation	python -m *src.run_crater_ml* executed	Random Forest baseline trains and exports the crater-probability raster	Initial issue was corrected; rerun generated crater_proba_ml_v1.npy and the associated model and summary outputs	Pass

Table 4 - Test case summary

7.2 Test methodology

This section explains how the outputs of the Endymion pipeline were verified and why those verification methods were appropriate for the project. Testing was not limited to checking whether the code executed without error. Instead, verification was carried out at three levels: component-level checks, end-to-end pipeline checks, and controlled benchmark comparison. This was necessary because Endymion is a modular analytical system in which correctness depends not only on individual functions, but also on the consistency of saved artefacts, hazard generation, pathfinding behaviour, and comparative evaluation across hazard variants.

7.2.1 Verification and validation approach

Verification was first performed at the component level. Individual stages, such as DEM loading, ROI resolution, terrain feature extraction, crater raster generation, hazard construction, cost-grid creation, pathfinding, and evaluation, were checked separately to confirm that each stage produced outputs of the expected form before being used by later stages. In practice, this included validation of raster shape and alignment, confirmation that cached artefacts matched the active ROI, and explicit rejection of invalid inputs such as missing files, oversized ROIs, or out-of-bounds coordinates. These checks were important because many later failures in the pipeline would otherwise be difficult to interpret if earlier spatial assumptions were already incorrect.

Figure 12 - Validation approach

Validation was then extended to end-to-end pipeline execution. Full runs were used to confirm that the official runner could move from DEM access through terrain or crater-aware hazard generation to navigation and final evaluation, while also saving the expected artefacts for traceability. At this level, validation focused on whether the system produced internally consistent outputs rather than merely completing execution. Hazard and cost rasters were checked for valid ranges and absence of unintended *NaN* values, path outputs were checked for successful or safely controlled failure states, and evaluator metrics were used to confirm that the generated route could be assessed quantitatively in terms of hazard exposure, efficiency, and search effort.

A further level of validation was then performed through visual and comparative inspection. Hazard maps, crater overlays, and path overlays were reviewed to confirm that outputs were spatially plausible and that route geometry behaved consistently with the hazard model. This did not replace quantitative testing, but it served as a useful secondary check when validating crater-aligned products and hazard-aware path deviation. In other words, quantitative metrics were used as the main evidence, while visual inspection was used to detect obviously implausible spatial behaviour that a single scalar metric might hide.

Finally, benchmark comparison was used as the main validation method for later-stage models. Rather than judging the crater-aware and ML-assisted stages in isolation, they were tested against the same cases and under the same planning conditions as the terrain-only baseline. This made it possible to distinguish between two separate questions: whether a model variant executed correctly, and whether it actually improved the safety-efficiency trade-off of the resulting route. That distinction was important in this project because some extended hazard variants were fully operational but still did not outperform the simpler baseline

7.2.2 Benchmark methodology

Benchmarking was treated as a controlled comparison exercise rather than as an open-ended optimisation task. The purpose of the benchmark stage was not simply to produce multiple routes, but to determine whether changing the hazard-generation stage altered navigation behaviour under fixed planning conditions. For that reason, the spatial context, benchmark cases, cost conversion, and pathfinding settings were held constant, while only the hazard representation was allowed to vary. This made it possible to compare terrain-only, crater-aware, and ML-assisted hazard variants as alternative components inside the same framework, rather than as unrelated standalone models

Element held constant	Reason	Value/description
Canonical ROI	Ensure all models are evaluated on the same spatial area	Fixed 1024 × 1024 canonical ROI for the selected south-polar DEM
Benchmark cases	Prevent route differences from being caused by different start-goal pairs	Same predefined benchmark scenarios reused across all model variants
Cost model	Ensure traversal penalties remain directly comparable	Same hazard-to-cost conversion, blocking threshold, and block cost
Pathfinder settings	Prevent search behaviour from changing independently of the hazard model	Same connectivity, heuristic weight, and corridor-radius strategy
Evaluator schema	Keep outcome metrics consistent across runs	Same metric definitions for hazard exposure, geometry, efficiency, and success/failure reporting

Table 5 - Summarises the benchmark elements

To reduce the risk of drawing conclusions from only one start-goal pair, benchmarking was carried out over a fixed set of six predefined cases within the same ROI. This provided a broader basis for comparison while still keeping the spatial context controlled.

Benchmark-set feature	Description	Purpose
Number of cases	Six fixed benchmark scenarios	Avoid judging model behaviour from only one route
Route variation	Mix of shorter and more demanding traversals	Observe behaviour under different search and terrain conditions
Case reuse	Same cases repeated for every hazard variant	Preserve fairness across terrain-only, crater-aware, and ML-assisted comparisons
Output structure	Per-case artefacts plus aggregated benchmark summaries	Support traceability, repeatability, and later inspection

Table 6 - Summarises the structure of the benchmark set.

Under this methodology, the benchmark process followed the same sequence for every hazard variant: the same ROI-derived terrain products were loaded, one hazard model was built, the full benchmark case set was executed, per-case outputs were saved, and the results were then aggregated into summary files. The same workflow was repeated for each model variant so that the resulting summaries could be compared directly.

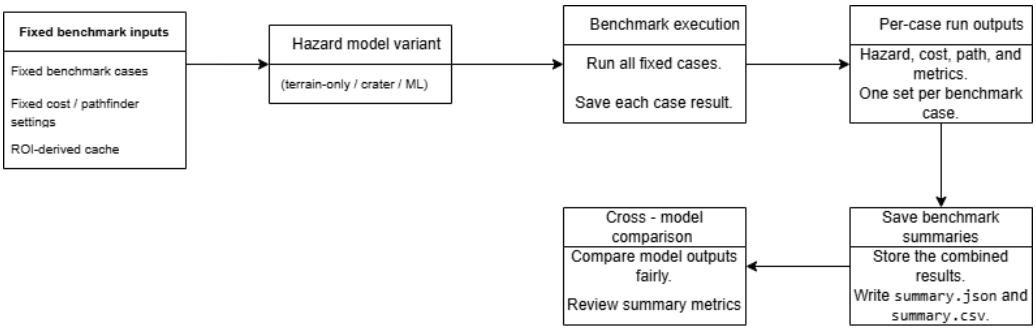


Figure 13 - Benchmark process

This workflow is useful because it shows clearly that only the hazard model changed between runs, while the benchmark contract itself remained fixed. As a result, the benchmark outputs functioned not only as evidence that the system executed successfully, but also as a repeatable basis for fair comparison across model variants.

A final three-model benchmark using this methodology showed that all six cases were solved successfully by the terrain-only, crater-aware, and ML-assisted variants. The terrain-only model remained the lightest in search effort, while the crater-informed variants reduced average hazard exposure and cost per metre, with the ML-assisted variant producing the lowest overall mean hazard and cost among the tested models.

(Representative benchmark CLI execution and summary outputs are provided in the appendix.)

7.2.3 Review and limitations

The testing methodology utilised was well-suited for a final-year project, as it effectively balanced technical rigour with a manageable scope. Component checks were performed to verify the accuracy of the main pipeline stages, while end-to-end runs confirmed the reproducibility of the official workflow. Additionally, benchmark comparisons provided a robust foundation for evaluating alternative hazard models. Collectively, these methods demonstrated that Endymion functioned as a modular and testable analytical pipeline, rather than merely as a one-off notebook prototype.

Nevertheless, the methodology has notable limitations. The benchmark set was intentionally small, the experiments concentrated on a single canonical region of interest (ROI), and large-scale validation across multiple lunar regions was beyond the project's final scope. Moreover, illumination modelling was deferred, meaning that the tested hazard variants primarily focused on terrain and crater-derived factors, rather than encompassing the full spectrum of environmental risks relevant to polar navigation. Therefore, the findings should be viewed as evidence from a controlled prototype rather than as definitive proof of generalised operational performance across the lunar south pole.

Some of these limitations were also shaped by practical development constraints rather than design choice alone. Progress was interrupted for a period of time by hardware and resource issues, while large dataset sizes and repository constraints limited the ability to version and work with heavier data products through the normal GitHub workflow. These constraints reduced the time available for further ML tuning, broader illumination integration, and wider comparative validation. In addition, project progress was affected by external personal circumstances, which further restricted the depth of late-stage experimentation. These factors do not invalidate the implemented results, but they do explain why the final submission prioritised a stable, reproducible analytical core over broader optimisation.

A further limitation is that some reviews remained based on technical inspection, internal consistency, and controlled benchmark outputs rather than on external mission-grade ground truth. This is acceptable for the scope of the project, but it means that the strongest claim supported by testing is not that Endymion has solved lunar navigation, but that it has established a reproducible framework in which hazard models can be integrated, executed, and compared under controlled conditions.

Because that framework is modular and based on general terrain-risk reasoning, it also has the potential to be adapted in future work to other extreme-terrain settings, including demanding Earth environments such as polar, mountainous, or low-visibility navigation scenarios; however, that transferability was not evaluated directly in this project

8. Conclusions and Reflections

The Endymion project set out to investigate whether lunar south-polar terrain data could be transformed into a usable hazard-aware navigation framework. Based on the final implementation, that objective was achieved. Over the course of development, the project moved from an early terrain-only prototype towards a more reproducible modular pipeline capable of loading a south-polar DEM, resolving a stable canonical ROI, generating terrain-derived products, incorporating crater-aware hazard information, executing hazard-aware pathfinding, and evaluating route quality through saved artefacts and benchmark summaries. In that sense, Endymion developed into more than a coding exercise alone, becoming a structured systems prototype with a clear analytical purpose.

One of the most important outcomes of the project was the shift from isolated experimentation towards controlled comparison. Early in development, the main concern was simply whether the system could transform terrain data into a route at all. Later, that focus moved towards reproducibility, modularity, and fair comparison between hazard-model variants. This progression mattered. The value of Endymion does not lie only in the fact that it can produce a path, but in the fact that it can compare terrain-only, crater-aware, and ML-assisted hazard variants under the same planning and evaluation contract. That is a more useful outcome than a single successful visual route, because it allows the system to support methodical analysis rather than demonstration alone.

At the same time, the final results were more nuanced than a simple narrative of steady improvement through added complexity. The terrain-only variant remained the strongest overall when judged in terms of simplicity, robustness, and lower search effort, while the ML-assisted

variant suggested that richer crater-aware signals could reduce average hazard exposure and cost per metre under the same benchmark contract. This was still an important result. It showed that the project did not depend on forcing the more advanced variants to appear superior, but instead preserved a comparison framework in which stronger and weaker approaches could be judged more honestly. In that respect, the project was valuable not because every extension outperformed the baseline, but because it made those differences visible and defensible through reproducible evidence.

Another significant outcome was architectural rather than numerical. A major part of the project's development involved moving away from a notebook-centred prototype towards a cleaner runner-based structure with explicit saved artefacts and clearer module responsibilities. That change mattered greatly. Without it, later crater integration, ML-assisted processing, and benchmark comparison would have remained difficult to reproduce and harder to justify academically. The project therefore reinforced an important lesson: for a system like Endymion, architecture was not separate from functionality, because reproducibility, modularity, and traceability were part of what made the implementation credible.

The project also made clear the limits of what could be achieved within the available scope. Endymion does not solve lunar south-polar navigation as a full operational mission system. The benchmark set remained intentionally limited, the experiments were centred on one canonical ROI, and illumination modelling, although recognised as important, was deferred so that the final submission could remain focused on a defensible analytical core. Likewise, the machine-learning stage was implemented as a baseline bridge rather than as a complete crater-recognition solution. These limitations do not undermine the project, but they do define the level of claim that can reasonably be made. Endymion therefore stands as a controlled research prototype and evaluation framework rather than a final mission-ready navigation product.

Some of those limits were methodological decisions, but others were shaped by practical realities during development. Parts of the project took longer than originally planned because the initial prototype had to be refactored into a more stable pipeline, the GeoTIFF-first workflow had to be stabilised, caching and artefact reuse needed to be formalised, and later crater-aware and ML-assisted stages had to be incorporated without weakening benchmark fairness. In addition, dataset size, repository limitations, and general development constraints reduced the time available for broader optimisation and wider experimentation. In retrospect, these difficulties were frustrating at the time, but they also pushed the project in a better direction by forcing greater attention onto stability, clarity, and methodological control.

A further reflection concerns the nature of the contribution itself. Endymion's contribution does not rest on one isolated algorithm or one especially novel model. Its stronger contribution lies in integration: taking terrain data, crater-aware reasoning, hazard modelling, constrained pathfinding, and evaluation, and combining them into a single framework that can be executed, tested, and compared in a repeatable way. That systems-level contribution is easy to overlook, but it is important, because much of the related work exists in separate pieces rather than as a unified hazard-navigation workflow.

The project also leaves several realistic directions for future work. The most immediate is illumination-aware hazard modelling, since lighting conditions are fundamental to polar traversal risk and were only partially represented in the final submission. A second direction is broader validation across additional ROIs and larger benchmark sets, which would strengthen claims about

generalisability. A third is refinement of the crater-prediction stage through alternative learners, richer feature engineering, or better-calibrated probability outputs. Beyond that, the architecture could support larger-scale extensions such as multi-tile traversal, more mission-oriented safety policies, or further postgraduate-level investigation into hazard modelling for extreme terrain environments. These are not guaranteed next steps, but they are realistic continuations because the final project established a modular basis on which such work could build.

Overall, Endymion resulted in a reproducible hazard-navigation framework built on real lunar south-polar terrain data, rather than only a one-off demonstration. Its strongest contribution lies not in claiming to solve lunar polar navigation outright, but in showing that the problem can be approached through a structured system that supports testing, comparison, and extension. By progressing from an early terrain-only prototype to a benchmarkable pipeline with crater-aware and ML-assisted extensions, the project developed into a more stable basis for further analysis and future work.

References

Barker, M.K., Mazarico, E., Neumann, G.A., Zuber, M.T., Haruyama, J. and Smith, D.E. (2016). A new lunar digital elevation model from the Lunar Orbiter Laser Altimeter and SELENE Terrain Camera. *Icarus*, [online] 273, pp.346–355. doi:10.1016/j.icarus.2015.07.039.

Bickel, V.T., Moseley, B., Lopez-Francos, I. and Shirley, M. (2021). Peering into lunar permanently shadowed regions with deep learning. *Nature Communications*, [online] 12(1). doi:10.1038/s41467-021-25882-z.

Chin, G., Brylow, S., Foote, M., Garvin, J., Kasper, J., Keller, J., Litvak, M., Mitrofanov, I., Paige, D., Raney, K., Robinson, M., Sanin, A., Smith, D., Spence, H., Spudis, P., Stern, S.A. and Zuber, M. (2007). Lunar Reconnaissance Orbiter Overview: The Instrument Suite and Mission. *Space Science Reviews*, [online] 129(4), pp.391–419. doi:10.1007/s11214-007-9153-y.

Deutsch, A.N., Head, J.W., Neumann, G.A., Kreslavsky, M.A. and Barker, M.K. (2020). Assessing the Roughness Properties of Circumpolar Lunar Craters: Implications for the Timing of Water-Ice Delivery to the Moon. *Geophysical Research Letters*, [online] 47(15). doi:10.1029/2020gl087782.

Giannakis, I., Bhardwaj, A., Sam, L. and Leontidis, G. (2024). A flexible deep learning crater detection scheme using Segment Anything Model (SAM). *Icarus*, [online] 408, p.115797. doi:10.1016/j.icarus.2023.115797.

Implementation Plan for a NASA Integrated Lunar Science Strategy in the Artemis Era. (2024). [online] Available at: https://ntrs.nasa.gov/api/citations/20240015059/downloads/Implementation_Plan_Final_2024.pdf .

Li, S., Lucey, P.G., Milliken, R.E., Hayne, P.O., Fisher, E., Williams, J.-P., Hurley, D.M. and Elphic, R.C. (2018). Direct evidence of surface exposed water ice in the lunar polar regions. *Proceedings of the National Academy of Sciences*, [online] 115(36), pp.8907–8912. doi:10.1073/pnas.1802345115.

McClanahan, T.P., Parsons, A.M., Livengood, T.A., Su, J.J., Chin, G., Hamara, D., Harshman, K. and Starr, R.D. (2024). Evidence for Widespread Hydrogen Sequestration within the Moon's South Pole Cold Traps. *The Planetary Science Journal*, 5(10), p.217. doi:10.3847/psj/ad5b55.

Pohl, I. (1970). Heuristic search viewed as path finding in a graph. *Artificial Intelligence*, 1(3–4), pp.193–204. doi:10.1016/0004-3702(70)90007-x.

Sinha, M. and Paul, S. (2025). Automated detection of craters on the lunar surface using deep learning: A review with insights from Chandrayaan-2 TMC-2 data. *Results in Earth Sciences*, [online] 3, p.100094. doi:10.1016/j.rines.2025.100094.

Smith, D.E., Zuber, M.T., Jackson, G.B., Cavanaugh, J.F., Neumann, G.A., Riris, H., Sun, X., Zellar, R.S., Coltharp, C., Connelly, J., Katz, R.B., Kleyner, I., Liiva, P., Matuszeski, A., Mazarico, E.M., McGarry, J.F., Novo-Gradac, A.-M., Ott, M.N., Peters, C., Ramos-Izquierdo, L.A., Ramsey, L., Rowlands, D.D., Schmidt, S., Scott, V.S., Shaw, G.B., Smith, J.C., Swinski, J.-P., Torrence, M.H., Unger, G., Yu, A.W. and Zagwodzki, T.W. (2009). The Lunar Orbiter Laser Altimeter Investigation on the Lunar Reconnaissance Orbiter Mission. *Space Science Reviews*, [online] 150(1–4), pp.209–241. doi:10.1007/s11214-009-9512-y.

Bibliography

Gillies, S. (2025). *rasterio Documentation Release 1.4.4*. [online] Available at: https://rasterio.readthedocs.io/_/downloads/en/stable/pdf/ [Accessed 20 Apr. 2026].

Rouault, E. (2026) 'GDAL'. Zenodo. doi:10.5281/zenodo.19133683.

Scikit-learn.org. (2025). *scikit-learn: machine learning in Python — scikit-learn 1.8.0 documentation*. [online] Available at: https://scikit-learn.org/stable/ [Accessed 20 Apr. 2026].

GitHub. (2025). *a-saga/lunar-landing-zone-detection-ai: AI-powered lunar safe landing zone generation system using YOLOv8 and DEM terrain fusion. Developed during research internship at ISRO.* [online] Available at: https://github.com/a-saga/lunar-landing-zone-detection-ai [Accessed 20 Apr. 2026].

GitHub. (2025). *Pirate-Emperor/Slope: Slope is a comprehensive repository developed by Pirate-Emperor that focuses on generating slope maps from various lunar models, including Lunar_ISR, Crater_Detection, Height_Map, DTM_3D, and Hazard_Map. This project combines the outputs of these models to create detailed slope maps of the lunar terrain.* [online] Available at: https://github.com/Pirate-Emperor/Slope [Accessed 20 Apr. 2026].

GitHub. (2025). *Pirate-Emperor/Crater_Detection: Crater_Detection is a specialized repository developed by Pirate-Emperor that focuses on identifying and outlining impact craters on lunar surfaces. This project combines the outputs of various lunar models, including Lunar_ISR, Crater_Detection itself, Height_Map, DTM_3D, Slope, Shadow, and Hazard_Map, to create detailed crater maps.* [online] Available at: https://github.com/Pirate-Emperor/Crater_Detection [Accessed 20 Apr. 2026].

GitHub. (2025). *Pirate-Emperor/Hazard_Map: Hazard_Map is a comprehensive repository developed by Pirate-Emperor that focuses on generating hazard maps from various lunar models, including Lunar_ISR, Crater_Detection, Height_Map, DTM_3D, Slope, Shadow, and Hazard_Map itself. This project combines the outputs of these models to create detailed hazard maps of the lunar terrain.* [online] Available at: https://github.com/Pirate-Emperor/Hazard_Map [Accessed 20 Apr. 2026].

GitHub. (2025). *bnsreenu/python_for_microscopists: https://www.youtube.com/channel/UC34rW-HtPJulxr5wp2Xa04w?sub_confirmation=1*. [online] Available at: https://github.com/bnsreenu/python_for_microscopists [Accessed 20 Apr. 2026].

Appendix

GitHub Repo: https://github.com/Log-Yair/Endymion/tree/phase2-ml-illum

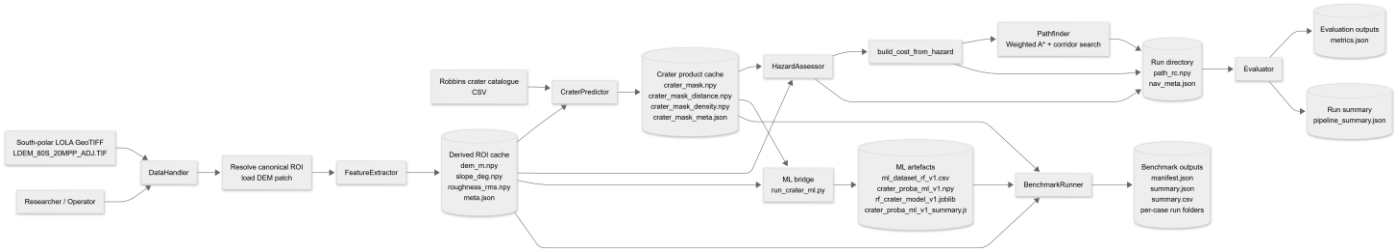
FVD : https://drive.google.com/file/d/15FQYDcorFsoY9LJe1KkgkvcaYeL241gZ/view?usp=sharing

Robbinsons csv: lunar_crater_database_robbins_2018.csv (manually download if it takes too long via url)

GEOTIFF tile: https://pgda.gsfc.nasa.gov/data/LOLA_20mpp/LDEM_80S_20MPP_ADJ.TIF

Section 5

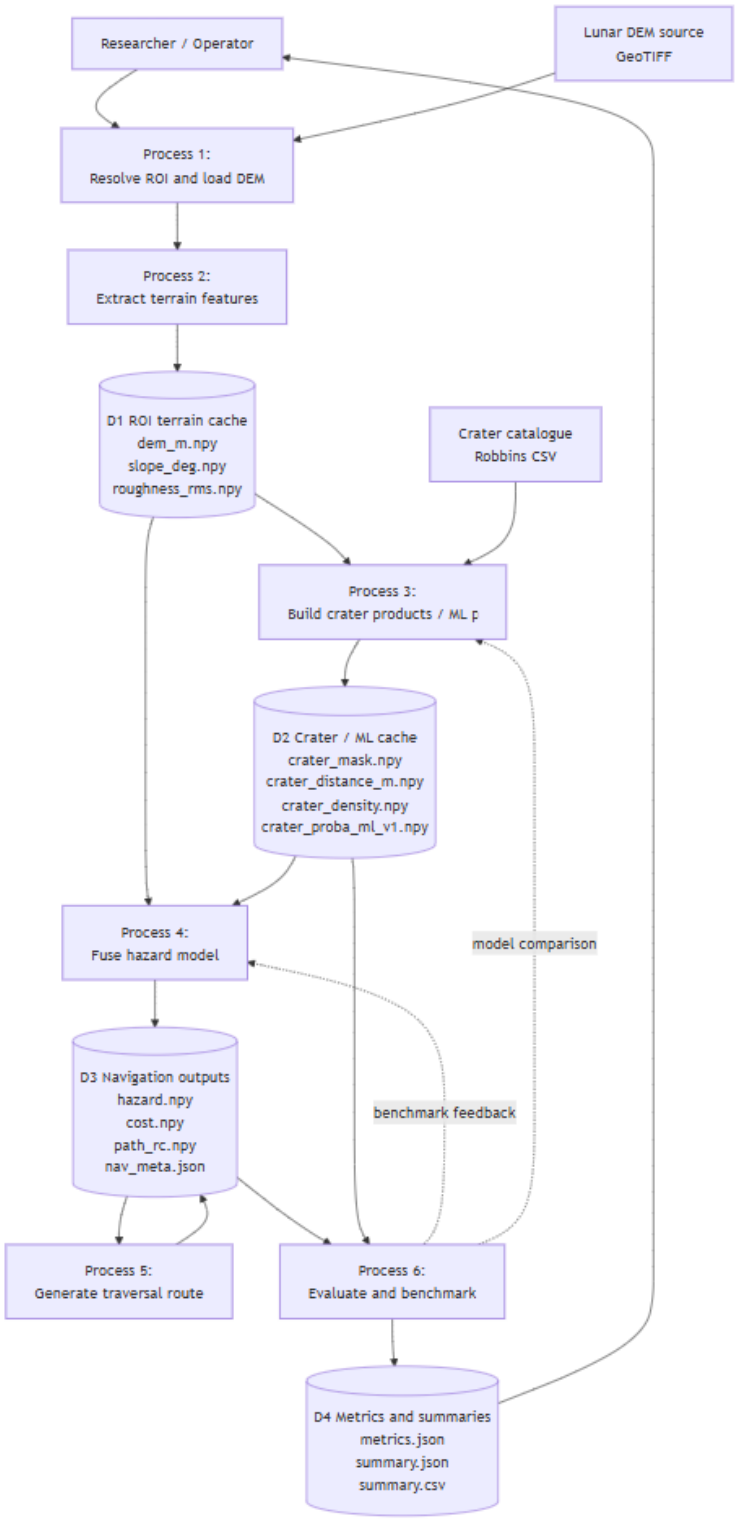
Appendix Section 5 - Detailed Data Flow Diagram of the saved artefacts



App Section 5 - extract showing build_cost_from_hazard(...) followed by corridor-aware Pathfinder

```python
def build_cost_from_hazard(
    hazard: np.ndarray,
    alpha: float = 10.0,
    hazard_block: float = 0.95,
    block_cost: float = 1e6,
) -> np.ndarray:
    """
    Convert hazard [0..1] into a traversal cost grid.

    cost = 1 + alpha * hazard

    Cells with hazard >= hazard_block become blocked (cost = block_cost).
    """
    if hazard.ndim != 2:
        raise ValueError("hazard must be a 2D raster.")
    if np.isnan(hazard).any():
        raise ValueError("hazard contains NaNs; fix/replace before building cost.")

    cost = (1.0 + float(alpha) * hazard).astype(np.float32)
    cost[hazard >= float(hazard_block)] = float(block_cost)
    return cost


@dataclass
class Pathfinder:
    """
    A* / Weighted A* over a 2D cost raster.

    Parameters
    ----------
    connectivity : int
        4 or 8 neighbours
    block_cost : float
        Any cell with cost >= block_cost is treated as blocked
    heuristic_weight : float
        1.0 = standard A* (optimal)
        >1.0 = weighted A* (faster, may be slightly suboptimal)
    """
    connectivity: int = 8
    block_cost: float = 1e6
    heuristic_weight: float = 1.2

    def find_path(
        self,
        cost: np.ndarray,
        start: RC,
        goal: RC,
        allowed_mask: Optional[np.ndarray] = None
    ) -> Dict[str, object]:
        """Compute a least-cost path from start to goal on a cost grid."""
        self._validate_inputs(cost, start, goal)

        if self.heuristic_weight < 1.0:
            raise ValueError("heuristic_weight must be >= 1.0 (1.0 = standard A*).")

        # Corridor mask sanity checks (optional feature)
        if allowed_mask is not None:
            if allowed_mask.shape != cost.shape:
                raise ValueError("allowed_mask must match cost shape.")
            if allowed_mask.dtype != bool:
```

App Section 5 - Endymion's final DFD



App Section 5 - Folder structure

```
ENDYMION
> .venv
> .vscode
> benchmark
> data
> db
> docs
> notebooks
> persistent
> src
.gitattributes
.gitignore
benchmark_cases_6.json                    U
lunar_crater_database_robbins_2018.csv
README.md
requirements.txt
```

```
ENDYMION
> .venv
> .vscode
v benchmark
  > __pycache__
  run_benchmark.py
> data
> db
v docs
  > diagrams
  > report
  README.md
v notebooks
  crater.ipynb
  Endymion_Exp_pipeline (4).ipynb
  Endymion_Exp_pipeline (5).ipynb
  Endymion_Exp_pipeline (7).ipynb
  Endymion_Exp_pipeline (11).ipynb
  Endymion_Exp_pipeline (12).ipynb
  Endymion_Exp_pipeline.ipynb
  Endymion_experimental_proto.ipynb
  Endymion_Prototype_Phase1_Demo.ipynb
  README.md
v persistent
  > derived\ldem_80s_20m\roi_14688_15712_1468...
  LDEM_80S_20MPP_ADJ.TIF
```

```
ENDYMION
v src
  > __pycache__
  v data
    > __pycache__
    __init__.py
    crater_raster.py
    data_handler.py
    README.md
  v evaluation
    > __pycache__
    __init__.py
    benchmark_runner.py
    evaluator.py
    README.md
  v features
    > __pycache__
    __init__.py
    feature_extractor.py
    README.md
  v models
    > __pycache__
    __init__.py
    crater_predictor.py
    hazard_assessor.py
    ml_dataset_builder.py
    README.md
  v planning
    > __pycache__
    __init__.py
    pathfinder.py
    README.md
  __init__.py
  README.md
  run_crater_ml.py
  run_endymion.py
.gitattributes
.gitignore
benchmark_cases_6.json                    U
lunar_crater_database_robbins_2018.csv
```

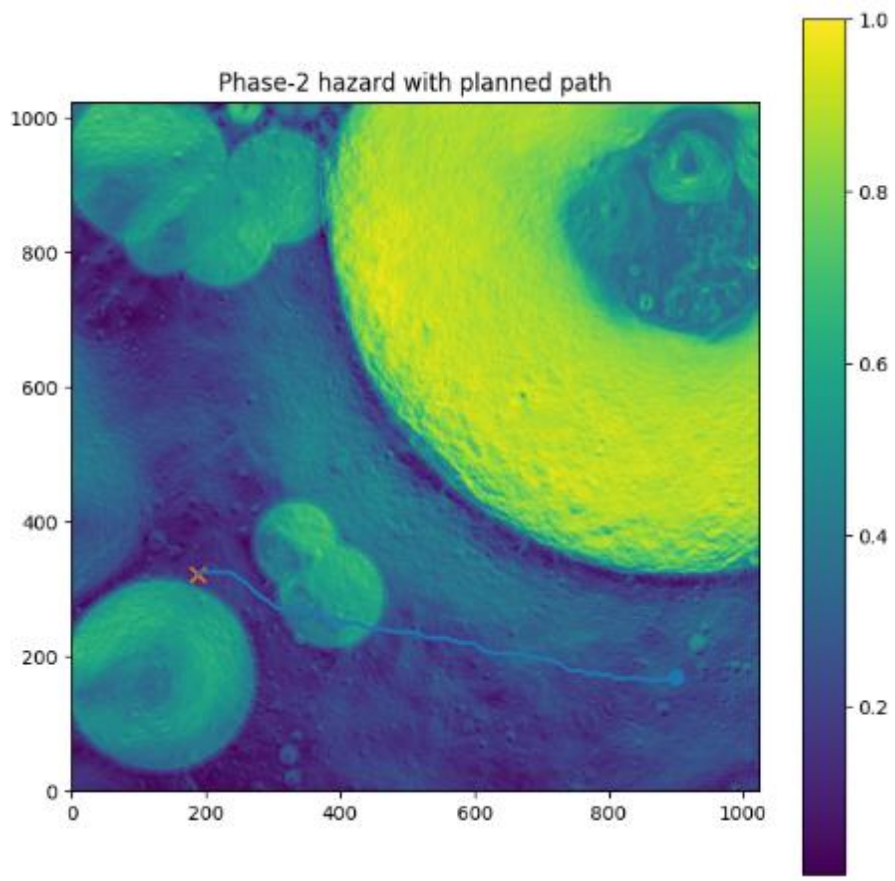
Section 6

App Section 6 - Extract of the FeatureExtractor output call and the saved derived products workflow.

```python
@dataclass
class FeatureExtractor:
    """
    Endymion FeatureExtractor (Phase 1):
    - Input: a DEM ROI (H x W) in meters.
    - Output: slope + roughness feature rasters aligned to the input grid.
    """
    pixel_size_m: float = 20.0
    slope_units: str = "degrees"  # "degrees" or "radians"
    expected_shape: Optional[Tuple[int, int]] = (1024, 1024)  # set None to disable
    strict_shape: bool = False

    def extract(
        self,
        dem_m: np.ndarray,
        roughness_method: RoughnessMethod = "rms",
        roughness_window: int = 5,
        pad_mode: Literal["edge", "reflect"] = "edge",
    ) -> Dict[str, np.ndarray]:
        """
        Returns a dict of feature rasters (float32):
          - slope_deg or slope_rad
          - slope_rise_run
          - roughness_<method>
        """
        self._validate_dem(dem_m)
        dem = dem_m.astype(np.float32, copy=False)

        dzdx, dzdy = self._gradients_central(dem, self.pixel_size_m, pad_mode=pad_mode)
        slope_rise_run = np.sqrt(dzdx * dzdx + dzdy * dzdy).astype(np.float32)

        if self.slope_units == "degrees":
            slope = np.degrees(np.arctan(slope_rise_run)).astype(np.float32)
            slope_key = "slope_deg"
        else:
            slope = np.arctan(slope_rise_run).astype(np.float32)
            slope_key = "slope_rad"

        if roughness_method == "rms":
            rough = self._roughness_rms(dem, window=roughness_window, pad_mode=pad_mode).astype(np.float32)
            rough_key = "roughness_rms"
        elif roughness_method == "tri":
            rough = self._roughness_tri(dem, pad_mode=pad_mode).astype(np.float32)
            rough_key = "roughness_tri"
        else:
            raise ValueError(f"Unknown roughness_method: {roughness_method}")

        return {
            slope_key: slope,
            "slope_rise_run": slope_rise_run,
            rough_key: rough,
        }

    def extract_from_dem(self, dem_m: np.ndarray, **kwargs) -> Dict[str, np.ndarray]:
        return self.extract(dem_m, **kwargs)

    # ------------------------
    # SLOPE
```

```python
def _load_or_build_derived(dh: DataHandler, cfg: RunConfig, roi: ROI) -> Dict[str, Any]:
    """Load terrain-derived rasters from cache or build them from the DEM."""
    if cfg.reuse_cached_derived:
        cached = dh.load_derived(cfg.tile_id, roi)
        if cached is not None:
            required = {"dem_m", "slope_deg", "roughness_rms"}
            if required.issubset(cached.keys()):
                return cached

    patch = dh.get_patch(cfg.tile_id, roi, verbose=True)
    dem_m = patch.data.astype(np.float32, copy=False)

    feature_extractor = FeatureExtractor(
        pixel_size_m=cfg.pixel_size_m,
        slope_units="degrees",
        expected_shape=dem_m.shape,
        strict_shape=False,
    )

    features = feature_extractor.extract(
        dem_m,
        roughness_method=cfg.roughness_method,
        roughness_window=cfg.roughness_window,
        pad_mode=cfg.pad_mode,
    )

    out_dir = dh.save_derived(
        cfg.tile_id,
        roi,
        dem_m=dem_m,
        features=features,
        meta_extra={
            "source_format": patch.meta.extra.get("source_format"),
            "pixel_size_m": cfg.pixel_size_m,
            "roughness_method": cfg.roughness_method,
            "roughness_window": cfg.roughness_window,
            "pad_mode": cfg.pad_mode,
            "patch_meta": patch.meta.extra,
        },
    )

    return {
        "meta": {"derived_dir": str(out_dir)},
        "dem_m": dem_m,
        **features,
    }
```

App Section 6 - Terrain-only hazard map for the canonical ROI



App Section 6 - HazardAssessor weighted - sum code extract

```python
# ------------------------------------------------------------
# Terrain component
# ------------------------------------------------------------
slope_n = self._normalise(slope_deg, vmax=self.slope_deg_max)
rough_n = self._normalise(roughness_rms, vmax=self.roughness_rms_max)

terrain_component = (
    self.w_slope * slope_n +
    self.w_roughness * rough_n
).astype(np.float32)

# ------------------------------------------------------------
# Crater component
# ------------------------------------------------------------
crater_inputs_present = any(
    arr is not None
    for arr in (crater_mask, crater_distance_m, crater_density)
)

if crater_inputs_present:
    crater_mask_n = self._prepare_crater_mask(
        crater_mask=crater_mask,
        expected_shape=slope_deg.shape,
    )

    crater_distance_risk = self._distance_to_risk(
        crater_distance_m=crater_distance_m,
        expected_shape=slope_deg.shape,
    )

    crater_density_n = self._prepare_crater_density(
        crater_density=crater_density,
        expected_shape=slope_deg.shape,
    )

    crater_component = (
        self.w_crater_mask * crater_mask_n +
        self.w_crater_distance * crater_distance_risk +
        self.w_crater_density * crater_density_n
    ).astype(np.float32)

    hazard = (
        self.terrain_weight * terrain_component +
        self.crater_weight * crater_component
    ).astype(np.float32)

    model_id = "terrain_crater_weighted_v2"

else:
    # Backwards-compatible terrain-only path
    crater_mask_n = np.zeros_like(slope_n, dtype=np.float32)
    crater_distance_risk = np.zeros_like(slope_n, dtype=np.float32)
    crater_density_n = np.zeros_like(slope_n, dtype=np.float32)
    crater_component = np.zeros_like(slope_n, dtype=np.float32)

    hazard = terrain_component.astype(np.float32)
    model_id = "terrain_weighted_v1_compatible"

# ------------------------------------------------------------
# Optional hard cutoff for extreme slopes
# ------------------------------------------------------------
if self.use_impassable_mask:
    hazard = hazard.copy()
    hazard[slope_deg > self.impassable_slope_deg] = 1.0

# Final safety clamp
hazard = np.clip(hazard, 0.0, 1.0).astype(np.float32)

# dem_m is currently unused, but we keep a small trace of whether it was supplied
dem_present = dem_m is not None
```

*App Section 6 - Cost grid creation + Weighted A execution**

```python
def build_cost_from_hazard(
    hazard: np.ndarray,
    alpha: float = 10.0,
    hazard_block: float = 0.95,
    block_cost: float = 1e6,
) -> np.ndarray:
    """
    Convert hazard [0..1] into a traversal cost grid.

    cost = 1 + alpha * hazard

    Cells with hazard >= hazard_block become blocked (cost = block_cost).
    """
    if hazard.ndim != 2:
        raise ValueError("hazard must be a 2D raster.")
    if np.isnan(hazard).any():
        raise ValueError("hazard contains NaNs; fix/replace before building cost.")

    cost = (1.0 + float(alpha) * hazard).astype(np.float32)
    cost[hazard >= float(hazard_block)] = float(block_cost)
    return cost
```

```python
def _run_pathfinding(cfg: RunConfig, cost: np.ndarray, hazard: np.ndarray) -> Dict[str, Any]:
    """Run corridor-aware pathfinding over the cost grid."""
    pathfinder = Pathfinder(
        connectivity=cfg.connectivity,
        block_cost=cfg.block_cost,
        heuristic_weight=cfg.heuristic_weight,
    )

    corr = pathfinder.find_path_corridor(
        cost=cost,
        start=tuple(int(x) for x in cfg.start_rc),
        goal=tuple(int(x) for x in cfg.goal_rc),
        corridor_radii=cfg.corridor_radii,
        hazard=hazard,
    )

    best = corr.get("best")
    success = bool(best is not None and best.get("success", False))

    if success:
        path_rc = np.asarray(best["path_rc"], dtype=np.int32)
        result_meta = best.get("meta", {})
        result = {
            "success": True,
            "failure_reason": None,
            "total_cost": best.get("total_cost"),
            "expansions": result_meta.get("expansions"),
        }
    else:
        path_rc = np.zeros((0, 2), dtype=np.int32)
        failure_reason = None
        if corr.get("experiment"):
            failure_reason = corr["experiment"][-1].get("reason")
        result = {
            "success": False,
            "failure_reason": failure_reason or "no_path",
            "total_cost": None,
            "expansions": None,
        }

    return {
        "success": success,
        "path_rc": path_rc,
        "corridor": corr,
        "result": result,
    }
```

App Section 6 - Module flow of run_endymion.py

```python
run_endymion.py ×
src > run_endymion.py > ...
# - System flow is based on the current Endymion architecture and project write-up:
#   DataHandler -> FeatureExtractor -> CraterPredictor -> HazardAssessor -> Pathfinder -> Evaluator.

# - Implementation contract follows the existing module/storage design already used in the project:
#   * DataHandler.save_derived(...) writes dem_m.npy / slope_deg.npy / roughness_rms.npy
#   * DataHandler.save_navigation(...) writes hazard.npy / cost.npy / path_rc.npy / nav_meta.json
#   * Evaluator then reads those artifacts and writes metrics.json

# - Design goal for this file:
#   keep one clean, official, reproducible pipeline runner separate from the benchmark runner
#   and separate from the presentation notebook.

# - Main ideas reused from the project files and notes:
#   * canonical ROI via DataHandler.canonical_roi(...)
#   * GeoTIFF-first DEM workflow
#   * crater catalogue cache/build path via CraterPredictor(catalogue_raster_v1)
#   * hazard -> cost via build_cost_from_hazard(...)
#   * corridor-aware Weighted A* via Pathfinder.find_path_corridor(...)
#   * post-run evaluation via Evaluator(...)
#
# - It supports both local GeoTIFF use and cache/download resolution.
# - It protects previous runs by auto-versioning repeated run names.
# - It validates ROI-local start/goal coordinates before pathfinding starts.

# Visual flow:
#
#   DEM / GeoTIFF
#       -> DataHandler
#       -> FeatureExtractor
#       -> CraterPredictor (optional)
#       -> HazardAssessor
#       -> build_cost_from_hazard
#       -> Pathfinder
#       -> DataHandler.save_navigation
#       -> Evaluator.save
#      -> pipeline_summary.json (with config, metrics, and important paths)
#
#   CLI arguments allow overriding all important config options, with sensible defaults for a canonical run.
#
#   The output directory structure is designed to be clean and navigable, with one run per subdirectory under the derived directory for the tile/ROI. Each run contains all relevant outputs and metadata, and a summary JSON for easy parsing.


# import statements are grouped into three sections:
# 1. Standard library imports
# 2. Third-party imports
# 3. Local application imports (with a fallback strategy for flat src aliases)

from __future__ import annotations

import argparse
import importlib
import json
import shutil
import sys
import types
from dataclasses import asdict, dataclass
from pathlib import Path
from time import perf_counter
from typing import Any, Dict, Optional, Sequence, Tuple

import numpy as np
```

```python
@dataclass
class RunConfig:
    """Configuration for one official Endymion pipeline run."""

    # Data source / storage
    base_url: str = "https://pgda.gsfc.nasa.gov/data/LOLA_20mpp"
    tile_id: str = "ldem_80s_20m"
    tif_filename: str = "LDEM_80S_20MPP_ADJ.TIF"
    tif_url: Optional[str] = None
    tif_path: Optional[str] = None  # If provided, a local GeoTIFF is used.
    persistent_dir: str = "./persistent"
    runtime_dir: Optional[str] = None
    allow_download: bool = True
    force_download: bool = False
    timeout_s: int = 120

    # ROI
    roi_size_px: int = 1024
    use_canonical_roi: bool = True
    roi: Optional[Tuple[int, int, int, int]] = None

    # Feature extraction
    pixel_size_m: float = 20.0
    roughness_method: str = "rms"
    roughness_window: int = 5
    pad_mode: str = "edge"
    reuse_cached_derived: bool = True

    # Crater products
    use_craters: bool = True
    crater_model_id: str = "catalogue_raster_v1"
    cache_product_name: str = "crater_mask"
    robbins_csv_path: Optional[str] = None
    rebuild_crater_if_missing: bool = False

    # Hazard model
    w_slope: float = 0.7
    w_roughness: float = 0.3
    slope_deg_max: float = 34.55
    roughness_rms_max: float = 18.56
    crater_distance_safe_m: float = 200.0
    crater_density_max: float = 0.25
    w_crater_mask: float = 0.4
    w_crater_distance: float = 0.4
    w_crater_density: float = 0.2
    terrain_weight: float = 0.7
    crater_weight: float = 0.3
    use_impassable_mask: bool = True
    impassable_slope_deg: float = 40.0

    # Cost model
    alpha: float = 10.0
    hazard_block: float = 0.95
    block_cost: float = 1e6

    # Pathfinding
    start_rc: RC = (269, 499)
    goal_rc: RC = (122, 558)
    connectivity: int = 8
    heuristic_weight: float = 1.2
    corridor_radii: Tuple[int, ...] = (25, 50, 80, 120, 180, 260, 400)

    # Output
    run_name: str = "final_run_v1.5"
    write_metrics: bool = True
```

App Section 6 - Phase 2 HazardAssessor fusion formula code extract.

```python
# ------------------------------------------------------------
# Terrain component
# ------------------------------------------------------------
slope_n = self._normalise(slope_deg, vmax=self.slope_deg_max)
rough_n = self._normalise(roughness_rms, vmax=self.roughness_rms_max)

terrain_component = (
    self.w_slope * slope_n +
    self.w_roughness * rough_n
).astype(np.float32)

# ------------------------------------------------------------
# Crater component
# ------------------------------------------------------------
crater_inputs_present = any(
    arr is not None
    for arr in (crater_mask, crater_distance_m, crater_density)
)

if crater_inputs_present:
    crater_mask_n = self._prepare_crater_mask(
        crater_mask=crater_mask,
        expected_shape=slope_deg.shape,
    )

    crater_distance_risk = self._distance_to_risk(
        crater_distance_m=crater_distance_m,
        expected_shape=slope_deg.shape,
    )

    crater_density_n = self._prepare_crater_density(
        crater_density=crater_density,
        expected_shape=slope_deg.shape,
    )

    crater_component = (
        self.w_crater_mask * crater_mask_n +
        self.w_crater_distance * crater_distance_risk +
        self.w_crater_density * crater_density_n
    ).astype(np.float32)

    hazard = (
        self.terrain_weight * terrain_component +
        self.crater_weight * crater_component
    ).astype(np.float32)

    model_id = "terrain_crater_weighted_v2"

else:
    # Backwards-compatible terrain-only path
    crater_mask_n = np.zeros_like(slope_n, dtype=np.float32)
    crater_distance_risk = np.zeros_like(slope_n, dtype=np.float32)
    crater_density_n = np.zeros_like(slope_n, dtype=np.float32)
    crater_component = np.zeros_like(slope_n, dtype=np.float32)

    hazard = terrain_component.astype(np.float32)
    model_id = "terrain_weighted_v1_compatible"

# ------------------------------------------------------------
# Optional hard cutoff for extreme slopes
# ------------------------------------------------------------
if self.use_impassable_mask:
    hazard = hazard.copy()
    hazard[slope_deg > self.impassable_slope_deg] = 1.0

# Final safety clamp
hazard = np.clip(hazard, 0.0, 1.0).astype(np.float32)

# dem_m is currently unused, but we keep a small trace of whether it was supplied
dem_present = dem_m is not None
```

App Section 6 - Weighted A corridor pathfinding call*

```python
def _run_pathfinding(cfg: RunConfig, cost: np.ndarray, hazard: np.ndarray) -> Dict[str, Any]:
    """Run corridor-aware pathfinding over the cost grid."""
    pathfinder = Pathfinder(
        connectivity=cfg.connectivity,
        block_cost=cfg.block_cost,
        heuristic_weight=cfg.heuristic_weight,
    )

    corr = pathfinder.find_path_corridor(
        cost=cost,
        start=tuple(int(x) for x in cfg.start_rc),
        goal=tuple(int(x) for x in cfg.goal_rc),
        corridor_radii=cfg.corridor_radii,
        hazard=hazard,
    )

    best = corr.get("best")
    success = bool(best is not None and best.get("success", False))

    if success:
        path_rc = np.asarray(best["path_rc"], dtype=np.int32)
        result_meta = best.get("meta", {})
        result = {
            "success": True,
            "failure_reason": None,
            "total_cost": best.get("total_cost"),
            "expansions": result_meta.get("expansions"),
        }
    else:
        path_rc = np.zeros((0, 2), dtype=np.int32)
        failure_reason = None
        if corr.get("experiment"):
            failure_reason = corr["experiment"][-1].get("reason")
        result = {
            "success": False,
            "failure_reason": failure_reason or "no_path",
            "total_cost": None,
            "expansions": None,
        }

    return {
        "success": success,
        "path_rc": path_rc,
        "corridor": corr,
        "result": result,
    }
```

```python
class Pathfinder:
    def find_path_corridor(
        start: RC,
        goal: RC,
        corridor_radii: Sequence[int],
        hazard: Optional[np.ndarray] = None,
    ) -> Dict[str, Any]:
        """
        Try progressively wider corridor masks until a path is found.

        Returns:
        - best: best path result (or None)
        - best_radius_px: radius that worked (or None)
        - experiment: list of per-radius logs
        """
        H, W = cost.shape
        experiment: List[Dict[str, Any]] = []

        best = None
        best_rad = None

        for rad in corridor_radii:
            allowed = make_corridor_mask((H, W), start, goal, radius_px=int(rad))

            # Fail fast if corridor doesn't contain endpoints
            if not allowed[start] or not allowed[goal]:
                experiment.append({
                    "corridor_radius_px": int(rad),
                    "success": False,
                    "expansions": None,
                    "path_len": 0,
                    "total_cost": None,
                    "reason": "start_or_goal_outside_corridor",
                })
                continue

            res = self.find_path(cost, start, goal, allowed_mask=allowed)

            total_cost = res.get("total_cost", None)
            if total_cost is not None and not np.isfinite(total_cost):
                total_cost = None

            row: Dict[str, Any] = {
                "corridor_radius_px": int(rad),
                "success": bool(res.get("success", False)),
                "expansions": res.get("meta", {}).get("expansions", None),
                "path_len": len(res.get("path_rc", [])),
                "total_cost": total_cost,
                "reason": res.get("meta", {}).get("reason", None),
            }
            if hazard is not None and row["success"] and row["path_len"] > 0:
                path_rc = np.asarray(res["path_rc"], dtype=np.int32)
                vals = hazard[path_rc[:, 0], path_rc[:, 1]]
                row["path_hazard_mean"] = float(np.mean(vals))
                row["path_hazard_max"] = float(np.max(vals))

            experiment.append(row)

            if row["success"] and row["path_len"] > 0:
                best = res
                best_rad = int(rad)
```

App Section 6 - Dataset-builder extract showing terrain features as predictors and crater mask as label

```python
# ----------------------------------------------------------------
# Core raster -> dataframe conversion
# ----------------------------------------------------------------
# This is where we flatten the aligned rasters into a per-pixel DataFrame, and add row/col and block IDs if requested.
def _rasters_to_dataframe(
    self,
    *,
    feature_rasters: Dict[str, np.ndarray],
    label: np.ndarray,
    include_rowcol: bool,
    include_block_ids: bool,
) -> pd.DataFrame:
    """
    Flatten aligned rasters into a per-pixel DataFrame.
    """
    ref_name, ref = next(iter(feature_rasters.items())) # take the first feature raster as reference for shape and alignment
    H, W = ref.shape

    rows, cols = np.indices((H, W), dtype=np.int32)

    data: Dict[str, np.ndarray] = {} # initialize data dict to collect columns for the DataFrame

    # Add row and col if requested
    if include_rowcol:
        data["row"] = rows.ravel()
        data["col"] = cols.ravel()

    # Add block IDs if requested (for spatial splitting later)
    if include_block_ids:
        block_row = (rows // self.block_size_px).astype(np.int32)
        block_col = (cols // self.block_size_px).astype(np.int32)
        block_id = (block_row * 100000 + block_col).astype(np.int32)

        data["block_row"] = block_row.ravel()
        data["block_col"] = block_col.ravel()
        data["block_id"] = block_id.ravel()

    # Add feature columns
    for name, arr in feature_rasters.items():
        self._validate_same_shape(name, arr, ref.shape)
        data[name] = arr.ravel()

    data[self.label_column] = label.astype(np.uint8, copy=False).ravel() # ensure label is binary uint8 and add to data dict

    return pd.DataFrame(data) # convert the collected data dict into a pandas DataFrame

    # ----------------------------------------------------------------
# Cleaning
# ----------------------------------------------------------------

def _drop_invalid_rows(self, df: pd.DataFrame, feature_columns: List[str]) -> pd.DataFrame:
    """
    Remove rows where any feature is NaN or infinite,since these would not be valid inputs for ML models.
    """
    valid_mask = np.ones(len(df), dtype=bool) #all true mask to start with

    # Check each feature column for finite values and update the valid_mask accordingly
    for col in feature_columns:
        vals = df[col].to_numpy()
        valid_mask &= np.isfinite(vals)

    return df.loc[valid_mask].copy() #copy any remaining valid rows into a new DataFrame and return it
```

App Section 6 - Runner extract showing model training and saving crater_proba_ml_v1.npy

```python
# ------------------------------------------------------------------------
# 1) Build dataset from existing ROI cache
# ------------------------------------------------------------------------
builder = MLDatasetBuilder(
    block_size_px=cfg.block_size_px,
    random_state=cfg.random_state,
)

log_step("Building ML dataset from ROI cache...")
dataset_out = builder.build_from_cache(
    dh=dh,
    tile_id=cfg.tile_id,
    roi=roi,
    include_rowcol=True,
    include_block_ids=True,
    balance_strategy="undersample",
    negative_ratio=cfg.negative_ratio,
    save_csv=cfg.save_dataset_csv,
    csv_name=cfg.dataset_csv_name,
)

df = dataset_out["dataframe"].copy()
meta = dataset_out["meta"]

feature_cols = ["elevation_m", "slope_deg", "roughness_rms"]
target_col = "label_crater"

required_cols = ["row", "col", "block_id", *feature_cols, target_col]
missing_cols = [c for c in required_cols if c not in df.columns]
if missing_cols:
    raise ValueError(f"Dataset is missing required columns: {missing_cols}")

if df.empty:
    raise ValueError("Built dataset is empty. Cannot train crater model.")

H, W = tuple(meta["shape"])

log_step(f"Dataset rows: {len(df):,}")
log_step(
    "Class counts: "
    f"positive={(df[target_col] == 1).sum():,}, "
    f"negative={(df[target_col] == 0).sum():,}"
)

# ------------------------------------------------------------------------
# 2) Spatial split using block_id (same spirit as notebook)
# ------------------------------------------------------------------------
unique_blocks = np.array(sorted(df["block_id"].unique()))
rng = np.random.default_rng(cfg.random_state)
rng.shuffle(unique_blocks)

n_test_blocks = max(1, int(len(unique_blocks) * cfg.test_fraction))
test_blocks = set(unique_blocks[:n_test_blocks])
train_blocks = set(unique_blocks[n_test_blocks:])

train_df = df[df["block_id"].isin(train_blocks)].copy()
test_df = df[df["block_id"].isin(test_blocks)].copy()

X_train = train_df[feature_cols]
y_train = train_df[target_col]
X_test = test_df[feature_cols]
y_test = test_df[target_col]

if len(train_df) == 0 or len(test_df) == 0:
    raise ValueError("Train/test split produced an empty subset. Adjust block size or test fraction.")

log_step(f"Train rows: {len(train_df):,}")
log_step(f"Test rows: {len(test_df):,}")
log_step(f"Train positive ratio: {float(y_train.mean()):.6f}")
log_step(f"Test positive ratio: {float(y_test.mean()):.6f}")
```

```python
# ------------------------------------------------------------------------
# 3) Train Random Forest baseline
# ------------------------------------------------------------------------
rf = RandomForestClassifier(
    n_estimators=cfg.n_estimators,
    max_depth=cfg.max_depth,
    min_samples_leaf=cfg.min_samples_leaf,
    class_weight=cfg.class_weight,
    random_state=cfg.random_state,
    n_jobs=cfg.n_jobs,
)

log_step("Training Random Forest baseline...")
rf.fit(X_train, y_train)

# ------------------------------------------------------------------------
# 4) Evaluate on held-out blocks
# ------------------------------------------------------------------------
y_pred = rf.predict(X_test)
y_proba = rf.predict_proba(X_test)[:, 1]

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, zero_division=0)
rec = recall_score(y_test, y_pred, zero_division=0)
f1 = f1_score(y_test, y_pred, zero_division=0)
cm = confusion_matrix(y_test, y_pred)
class_report = _classification_report_dict(y_test.to_numpy(), y_pred)

log_step(
    f"Held-out metrics: accuracy={acc:.4f}, precision={prec:.4f}, "
    f"recall={rec:.4f}, f1={f1:.4f}"
)

# ------------------------------------------------------------------------
# 5) Feature importances
# ------------------------------------------------------------------------
importance_df = pd.DataFrame(
    {
        "feature": feature_cols,
        "importance": rf.feature_importances_,
    }
).sort_values("importance", ascending=False)

# ------------------------------------------------------------------------
# 6) Predict crater probability for the full ROI
# ------------------------------------------------------------------------
log_step("Predicting full ROI crater probabilities...")
X_all = df[feature_cols]
all_proba = rf.predict_proba(X_all)[:, 1]

crater_proba_ml = np.full((H, W), np.nan, dtype=np.float32)
rows = df["row"].to_numpy(dtype=int)
cols = df["col"].to_numpy(dtype=int)
crater_proba_ml[rows, cols] = all_proba.astype(np.float32)

# ------------------------------------------------------------------------
# 7) Save outputs
# ------------------------------------------------------------------------
product_path = derived_dir / f"{cfg.product_name}.npy"
np.save(product_path, crater_proba_ml)

model_path = derived_dir / cfg.model_filename
saved_model_path = _save_model(rf, model_path)

feature_importance_path = derived_dir / cfg.feature_importance_filename
feature_importance_path.write_text(
    json.dumps(importance_df.to_dict(orient="records"), indent=2)
)
```

```python
summary = {
    "config": asdict(cfg),
    "tile_id": cfg.tile_id,
    "roi": list(roi),
    "dataset": {
        "rows": int(len(df)),
        "shape": [int(H), int(W)],
        "feature_columns": feature_cols,
        "label_column": target_col,
        "positive_count": int((df[target_col] == 1).sum()),
        "negative_count": int((df[target_col] == 0).sum()),
        "positive_ratio": float(df[target_col].mean()),
        "builder_meta": meta,
        "builder_paths": dataset_out.get("paths", {}),
    },
    "split": {
        "test_fraction": float(cfg.test_fraction),
        "num_unique_blocks": int(len(unique_blocks)),
        "num_test_blocks": int(n_test_blocks),
        "train_rows": int(len(train_df)),
        "test_rows": int(len(test_df)),
        "train_positive_ratio": float(y_train.mean()),
        "test_positive_ratio": float(y_test.mean()),
    },
    "model": {
        "type": "RandomForestClassifier",
        "n_estimators": int(cfg.n_estimators),
        "max_depth": int(cfg.max_depth),
        "min_samples_leaf": int(cfg.min_samples_leaf),
        "class_weight": cfg.class_weight,
    },
    "evaluation": {
        "accuracy": float(acc),
        "precision": float(prec),
        "recall": float(rec),
        "f1": float(f1),
        "confusion_matrix": cm.tolist(),
        "classification_report": class_report,
    },
    "feature_importances": importance_df.to_dict(orient="records"),
    "probability_raster_stats": {
        "min": float(np.nanmin(crater_proba_ml)),
        "max": float(np.nanmax(crater_proba_ml)),
        "mean": float(np.nanmean(crater_proba_ml)),
    },
    "outputs": {
        "probability_raster": str(product_path),
        "model": saved_model_path,
        "feature_importances": str(feature_importance_path),
    },
}

summary_path = derived_dir / cfg.summary_filename
summary_path.write_text(json.dumps(summary, indent=2))

log_path("Saved probability raster", product_path)
log_path("Saved model", saved_model_path)
log_path("Saved feature importances", feature_importance_path)
log_path("Saved summary", summary_path)

return {
    "summary": summary,
    "summary_path": summary_path,
    "product_path": product_path,
    "derived_dir": derived_dir,
}
```

Section 7

App. Fig. T1. 1

```
(.venv) PS C:\Users\yairc\Endymion\Endymion>
>> python -m src.run_endymion `
>>   --persistent-dir $PERSISTENT `
>>   --runtime-dir $RUNTIME `
>>   --tif-path $TIF `
>>   --run-name app_t1_1 `
>>   --no-craters `
>>   --no-metrics
[Endymion] Initialising run...
[Endymion] Persistent dir: C:\Endymion\persistent
[Endymion] Runtime dir: C:\Endymion\runtime
[Endymion] Local GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[Endymion] Cached GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[Endymion] Initialising DataHandler...
[Endymion] Resolving ROI...
[DataHandler] Resolving canonical ROI for tile 'ldem_80s_20m' with size 1024...
[DataHandler] Resolving GeoTIFF for tile 'ldem_80s_20m'...
[DataHandler] GeoTIFF found in runtime cache: C:\Endymion\runtime\LDEM_80S_20MPP_ADJ.TIF
[DataHandler] Opening GeoTIFF to read raster shape: C:\Endymion\runtime\LDEM_80S_20MPP_ADJ.TIF
[DataHandler] Raster shape resolved: (30400, 30400)
[DataHandler] Canonical ROI resolved: (14688, 15712, 14688, 15712)
[Endymion] Using ROI: (14688, 15712, 14688, 15712)
[Endymion] Loading or building derived terrain rasters...
[Endymion] Derived terrain ready in 0.04s
[Endymion] Skipping crater products (--no-craters enabled).
[Endymion] Building hazard map...
[Endymion] Hazard map ready in 0.03s
[Endymion] Building traversal cost grid...
[Endymion] Cost grid ready in 0.01s
[Endymion] Running pathfinder...
[Endymion] Pathfinder finished in 0.15s
[Endymion] Saving navigation outputs...
[Endymion] Run dir: C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_1
[Endymion] Skipping evaluator (--no-metrics enabled).
[Endymion] Run complete.
==================================================================
Endymion run complete
==================================================================
Run dir      : C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_1
Derived dir  : C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712
ROI          : (14688, 15712, 14688, 15712)
Run name     : app_t1_1
Success      : True
Failure      : None
Total cost   : 636.4488525390625
Expansions   : 8807
```

App. Fig. T1.2

```
>> python -m src.run_endymion  4688, 15712)
>>   --persistent-dir $PERSISTENT `
>>   --runtime-dir $RUNTIME `
>>   --tif-path "C:\missing\LDEM_80S_20MPP_ADJ.TIF" `
>>   --run-name app_t1_2 `390625
>>   --no-craters `
>>   --no-metricsrs\yairc\Endymion\Endymion>
[Endymion] Initialising run...
[Endymion] Persistent dir: C:\Endymion\persistent
[Endymion] Runtime dir: C:\Endymion\runtime
[run_endymion] ERROR: FileNotFoundError: Local GeoTIFF not found: C:\missing\LDEM_80S_20MPP_ADJ.TIF
(.venv) PS C:\Users\yairc\Endymion\Endymion>
```

App. Fig. T1.3

```
>> python -m src.run_endymion `
>>   --persistent-dir $PERSISTENT `
>>   --runtime-dir $RUNTIME `
>>   --tif-path $TIF `
>>   --run-name app_t1_3 `
>>   --no-canonical-roi `
>>   --roi 14688 15712 14688 15712 `
>>   --no-craters `
>>   --no-metrics
[Endymion] Initialising run...
[Endymion] Persistent dir: C:\Endymion\persistent
[Endymion] Runtime dir: C:\Endymion\runtime
[Endymion] Local GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[Endymion] Cached GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[Endymion] Initialising DataHandler...
[Endymion] Resolving ROI...
[Endymion] Using ROI: (14688, 15712, 14688, 15712)
[Endymion] Loading or building derived terrain rasters...
[Endymion] Derived terrain ready in 0.01s
[Endymion] Skipping crater products (--no-craters enabled).
[Endymion] Building hazard map...
[Endymion] Hazard map ready in 0.03s
[Endymion] Building traversal cost grid...
[Endymion] Cost grid ready in 0.01s
[Endymion] Running pathfinder...
[Endymion] Pathfinder finished in 0.14s
[Endymion] Saving navigation outputs...
[Endymion] Run dir: C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_3
[Endymion] Skipping evaluator (--no-metrics enabled).
[Endymion] Run complete.
====================================================================
Endymion run complete
====================================================================
Run dir      : C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_3
Derived dir  : C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712
ROI          : (14688, 15712, 14688, 15712)
Run name     : app_t1_3
Success      : True
Failure      : None
Total cost   : 636.4488525390625
Expansions   : 8807
```

App. Fig. T1.4

```
>> python -m src.run_endymion
>>   --persistent-dir $PERSISTENT `
>>   --runtime-dir $RUNTIME `
>>   --tif-path $TIF `
>>   --run-name app_t1_4 `
>>   --roi-size 50000 `
>>   --no-craters `
>>   --no-metrics
[Endymion] Initialising run...
[Endymion] Persistent dir: C:\Endymion\persistent
[Endymion] Runtime dir: C:\Endymion\runtime
[Endymion] Local GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[Endymion] Cached GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[Endymion] Initialising DataHandler...
[Endymion] Resolving ROI...
[DataHandler] Resolving canonical ROI for tile 'ldem_80s_20m' with size 50000...
[DataHandler] Resolving GeoTIFF for tile 'ldem_80s_20m'...
[DataHandler] GeoTIFF found in runtime cache: C:\Endymion\runtime\LDEM_80S_20MPP_ADJ.TIF
[DataHandler] Opening GeoTIFF to read raster shape: C:\Endymion\runtime\LDEM_80S_20MPP_ADJ.TIF
[DataHandler] Raster shape resolved: (30400, 30400)
[run_endymion] ERROR: ValueError: Requested size 50000 exceeds raster shape (30400, 30400)
```

App. Fig. T1.5

```
>> python -m src.run_endymion `
>>   --persistent-dir $PERSISTENT `
>>   --runtime-dir $RUNTIME `
>>   --tif-path $TIF `
>>   --run-name app_t1_5 `
>>   --rebuild-derived `
>>   --no-craters `
>>   --no-metrics
[Endymion] Initialising run...
[Endymion] Persistent dir: C:\Endymion\persistent
[Endymion] Runtime dir: C:\Endymion\runtime
[Endymion] Local GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[Endymion] Cached GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[Endymion] Initialising DataHandler...
[Endymion] Resolving ROI...
[DataHandler] Resolving canonical ROI for tile 'ldem_80s_20m' with size 1024...
[DataHandler] Resolving GeoTIFF for tile 'ldem_80s_20m'...
[DataHandler] GeoTIFF found in runtime cache: C:\Endymion\runtime\LDEM_80S_20MPP_ADJ.TIF
[DataHandler] Opening GeoTIFF to read raster shape: C:\Endymion\runtime\LDEM_80S_20MPP_ADJ.TIF
[DataHandler] Raster shape resolved: (30400, 30400)
[DataHandler] Canonical ROI resolved: (14688, 15712, 14688, 15712)
[Endymion] Using ROI: (14688, 15712, 14688, 15712)
[Endymion] Loading or building derived terrain rasters...
[DataHandler] GeoTIFF found in runtime cache: C:\Endymion\runtime\LDEM_80S_20MPP_ADJ.TIF
[Endymion] Derived terrain ready in 0.26s
[Endymion] Skipping crater products (--no-craters enabled).
[Endymion] Building hazard map...
[Endymion] Hazard map ready in 0.04s
[Endymion] Building traversal cost grid...
[Endymion] Cost grid ready in 0.00s
[Endymion] Running pathfinder...
[Endymion] Pathfinder finished in 0.14s
[Endymion] Saving navigation outputs...
[Endymion] Run dir: C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_5
[Endymion] Skipping evaluator (--no-metrics enabled).
[Endymion] Run complete.
======================================================================
Endymion run complete
======================================================================
Run dir      : C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_5
Derived dir  : C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712
ROI          : (14688, 15712, 14688, 15712)
Run name     : app_t1_5
Success      : True
Failure      : None
Total cost   : 636.4488525390625
Expansions   : 8807
```

App. Fig. T1.6

```
>> python -m src.run_endymion `
>>   --persistent-dir $PERSISTENT `
>>   --runtime-dir $RUNTIME `
>>   --tif-path $TIF `
>>   --run-name app_t1_6 `
>>   --no-canonical-roi `
>>   --roi 14688 15712 14688 15712 `
>>   --robbins-csv $ROBBINS `
>>   --rebuild-crater-if-missing
[Endymion] Initialising run...
[Endymion] Persistent dir: C:\Endymion\persistent
[Endymion] Runtime dir: C:\Endymion\runtime
[Endymion] Local GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[Endymion] Cached GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[Endymion] Initialising DataHandler...
[Endymion] Resolving ROI...
[Endymion] Using ROI: (14688, 15712, 14688, 15712)
[Endymion] Crater cache found in derived directory.
[Endymion] Loading or building derived terrain rasters...
[Endymion] Derived terrain ready in 0.03s
[Endymion] Loading or building crater products...
[Endymion] Crater products ready in 3.41s
[Endymion] Building hazard map...
[Endymion] Hazard map ready in 0.05s
[Endymion] Building traversal cost grid...
[Endymion] Cost grid ready in 0.00s
[Endymion] Running pathfinder...
[Endymion] Pathfinder finished in 0.13s
[Endymion] Saving navigation outputs...
[Endymion] Run dir: C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_6
[Endymion] Running evaluator...
[Endymion] Metrics path: C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_6\metrics.json
[Endymion] Run complete.
======================================================================
Endymion run complete
======================================================================
Run dir      : C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_6
Derived dir  : C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712
ROI          : (14688, 15712, 14688, 15712)
Run name     : app_t1_6
Success      : True
Failure      : None
Total cost   : 498.1625671386719
Expansions   : 8747
----------------------------------------------------------------------
Path length m: 3499.0663761154815
Mean hazard  : 0.1868754866375745
Max hazard   : 0.3050569295831787
Safety score : 0.9918423787603952
Cost / m     : 0.1262588935241243
Metrics path : C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_6\metrics.json
```

App. Fig. T1.7

```
>> python -m src.run_endymion `
>>   --persistent-dir $PERSISTENT `
>>   --runtime-dir $RUNTIME `
>>   --tif-path $TIF `
>>   --run-name app_t1_7 `
>>   --no-canonical-roi `
>>   --roi 14688 15712 14688 15712 `
>>   --robbins-csv "C:\missing\Robbins_catalogue.csv" `
>>   --rebuild-crater-if-missing
[Endymion] Initialising run...
[Endymion] Persistent dir: C:\Endymion\persistent
[Endymion] Runtime dir: C:\Endymion\runtime
[Endymion] Local GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[Endymion] Cached GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[Endymion] Initialising DataHandler...
[Endymion] Resolving ROI...
[Endymion] Using ROI: (14688, 15712, 14688, 15712)
[Endymion] Crater cache found in derived directory.
[Endymion] Loading or building derived terrain rasters...
[Endymion] Derived terrain ready in 0.01s
[Endymion] Loading or building crater products...
[run_endymion] ERROR: FileNotFoundError: Robbins crater catalogue CSV not found: C:\missing\Robbins_catalogue.csv
```

App. Fig. T1.8

```
>> python -m src.run_endymion `
>>   --persistent-dir $PERSISTENT `
>>   --runtime-dir $RUNTIME `
>>   --tif-path $TIF `
>>   --run-name app_t1_8 `
>>   --no-craters
[Endymion] Initialising run...
[Endymion] Persistent dir: C:\Endymion\persistent
[Endymion] Runtime dir: C:\Endymion\runtime
[Endymion] Local GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[Endymion] Cached GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[Endymion] Initialising DataHandler...
[Endymion] Resolving ROI...
[DataHandler] Resolving canonical ROI for tile 'ldem_80s_20m' with size 1024...
[DataHandler] Resolving GeoTIFF for tile 'ldem_80s_20m'...
[DataHandler] GeoTIFF found in runtime cache: C:\Endymion\runtime\LDEM_80S_20MPP_ADJ.TIF
[DataHandler] Opening GeoTIFF to read raster shape: C:\Endymion\runtime\LDEM_80S_20MPP_ADJ.TIF
[DataHandler] Raster shape resolved: (30400, 30400)
[DataHandler] Canonical ROI resolved: (14688, 15712, 14688, 15712)
[Endymion] Using ROI: (14688, 15712, 14688, 15712)
[Endymion] Loading or building derived terrain rasters...
[Endymion] Derived terrain ready in 0.01s
[Endymion] Skipping crater products (--no-craters enabled).
[Endymion] Building hazard map...
[Endymion] Hazard map ready in 0.03s
[Endymion] Building traversal cost grid...
[Endymion] Cost grid ready in 0.00s
[Endymion] Running pathfinder...
[Endymion] Pathfinder finished in 0.13s
[Endymion] Saving navigation outputs...
[Endymion] Run dir: C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_8
[Endymion] Running evaluator...
[Endymion] Metrics path: C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_8\metrics.json
[Endymion] Run complete.
======================================================================
Endymion run complete
======================================================================
Run dir      : C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_8
Derived dir  : C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712
ROI          : (14688, 15712, 14688, 15712)
Run name     : app_t1_8
Success      : True
Failure      : None
Total cost   : 636.4488525390625
Expansions   : 8807
----------------------------------------------------------------------
Path length m: 3510.7821048680194
Mean hazard  : 0.2659058178985311
Max hazard   : 0.4357956051826477
Safety score : 0.9883965566706623
Cost / m     : 0.16154634529362283
Metrics path : C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_8\metrics.json
(.venv) PS C:\Users\vairc\Endymion\Endymion>
```

App. Fig. T1.9

```
>> python -m src.run_endymion `
>>   --persistent-dir $PERSISTENT `
>>   --runtime-dir $RUNTIME `
>>   --tif-path $TIF `
>>   --run-name app_t1_9 `
>>   --no-craters `
>>   --start 269 499 `
>>   --goal 122 558
[Endymion] Initialising run...
[Endymion] Persistent dir: C:\Endymion\persistent
[Endymion] Runtime dir: C:\Endymion\runtime
[Endymion] Local GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[Endymion] Cached GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[Endymion] Initialising DataHandler...
[Endymion] Resolving ROI...
[DataHandler] Resolving canonical ROI for tile 'ldem_80s_20m' with size 1024...
[DataHandler] Resolving GeoTIFF for tile 'ldem_80s_20m'...
[DataHandler] GeoTIFF found in runtime cache: C:\Endymion\runtime\LDEM_80S_20MPP_ADJ.TIF
[DataHandler] Opening GeoTIFF to read raster shape: C:\Endymion\runtime\LDEM_80S_20MPP_ADJ.TIF
[DataHandler] Raster shape resolved: (30400, 30400)
[DataHandler] Canonical ROI resolved: (14688, 15712, 14688, 15712)
[Endymion] Using ROI: (14688, 15712, 14688, 15712)
[Endymion] Loading or building derived terrain rasters...
[Endymion] Derived terrain ready in 0.01s
[Endymion] Skipping crater products (--no-craters enabled).
[Endymion] Building hazard map...
[Endymion] Hazard map ready in 0.03s
[Endymion] Building traversal cost grid...
[Endymion] Cost grid ready in 0.00s
[Endymion] Running pathfinder...
[Endymion] Pathfinder finished in 0.13s
[Endymion] Saving navigation outputs...
[Endymion] Run dir: C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_9
[Endymion] Running evaluator...
[Endymion] Metrics path: C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_9\metrics.json
[Endymion] Run complete.
======================================================================
Endymion run complete
======================================================================
Run dir      : C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_9
Derived dir  : C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712
ROI          : (14688, 15712, 14688, 15712)
Run name     : app_t1_9
Success      : True
Failure      : None
Total cost   : 636.4488525390625
Expansions   : 8807
----------------------------------------------------------------------
Path length m: 3510.7821048680194
Mean hazard  : 0.2659058178985311
Max hazard   : 0.4357956051826477
Safety score : 0.9883965566706623
Cost / m     : 0.16154634529362283
Metrics path : C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_9\metrics.json
```

App. Fig. T1.10

```
>> python -m src.run_endymion `
>>   --persistent-dir $PERSISTENT `
>>   --runtime-dir $RUNTIME `
>>   --tif-path $TIF `
>>   --run-name app_t1_10 `
>>   --no-craters `
>>   --start 2000 2000 `
>>   --goal 122 558
[Endymion] Initialising run...
[Endymion] Persistent dir: C:\Endymion\persistent
[Endymion] Runtime dir: C:\Endymion\runtime
[Endymion] Local GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[Endymion] Cached GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[Endymion] Initialising DataHandler...
[Endymion] Resolving ROI...
[DataHandler] Resolving canonical ROI for tile 'ldem_80s_20m' with size 1024...
[DataHandler] Resolving GeoTIFF for tile 'ldem_80s_20m'...
[DataHandler] GeoTIFF found in runtime cache: C:\Endymion\runtime\LDEM_80S_20MPP_ADJ.TIF
[DataHandler] Opening GeoTIFF to read raster shape: C:\Endymion\runtime\LDEM_80S_20MPP_ADJ.TIF
[DataHandler] Raster shape resolved: (30400, 30400)
[DataHandler] Canonical ROI resolved: (14688, 15712, 14688, 15712)
[Endymion] Using ROI: (14688, 15712, 14688, 15712)
[Endymion] Loading or building derived terrain rasters...
[Endymion] Derived terrain ready in 0.01s
[run_endymion] ERROR: ValueError: Start point (2000, 2000) is out of bounds for ROI shape (1024, 1024). Valid row range: 0 to 1023; valid col range: 0 to 1023. The resolved ROI is (14688, 15712, 14688, 15712). Coordinates must be ROI-local, not full-raster pixel c
oordinates
```

App. Fig. T1.11

```
>> python -m src.run_endymion `
>>   --persistent-dir $PERSISTENT `
>>   --runtime-dir $RUNTIME `
>>   --tif-path $TIF `
>>   --run-name app_t1_11 `
>>   --no-craters `
>>   --hazard-block 0.0
[Endymion] Initialising run...
[Endymion] Persistent dir: C:\Endymion\persistent
[Endymion] Runtime dir: C:\Endymion\runtime
[Endymion] Local GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[Endymion] Cached GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[Endymion] Initialising DataHandler...
[Endymion] Resolving ROI...
[DataHandler] Resolving canonical ROI for tile 'ldem_80s_20m' with size 1024...
[DataHandler] Resolving GeoTIFF for tile 'ldem_80s_20m'...
[DataHandler] GeoTIFF found in runtime cache: C:\Endymion\runtime\LDEM_80S_20MPP_ADJ.TIF
[DataHandler] Opening GeoTIFF to read raster shape: C:\Endymion\runtime\LDEM_80S_20MPP_ADJ.TIF
[DataHandler] Raster shape resolved: (30400, 30400)
[DataHandler] Canonical ROI resolved: (14688, 15712, 14688, 15712)
[Endymion] Using ROI: (14688, 15712, 14688, 15712)
[Endymion] Loading or building derived terrain rasters...
[Endymion] Derived terrain ready in 0.01s
[Endymion] Skipping crater products (--no-craters enabled).
[Endymion] Building hazard map...
[Endymion] Hazard map ready in 0.03s
[Endymion] Building traversal cost grid...
[Endymion] Cost grid ready in 0.01s
[Endymion] Running pathfinder...
[Endymion] Pathfinder finished in 0.20s
[Endymion] Saving navigation outputs...
[Endymion] Run dir: C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_11
[Endymion] Running evaluator...
[Endymion] Metrics path: C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_11\metrics.json
[Endymion] Run complete.
==================================================================
Endymion run complete
==================================================================
Run dir      : C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_11
Derived dir  : C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712
ROI          : (14688, 15712, 14688, 15712)
Run name     : app_t1_11
Success      : False
Failure      : start_or_goal_blocked
Total cost   : None
Expansions   : None
------------------------------------------------------------------
Path length m: 0.0
Mean hazard  : None
Max hazard   : None
Safety score : None
Cost / m     : None
Metrics path : C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_11\metrics.json
```

App. Fig. T1.12

```
>> python -m src.run_crater_ml
>>   --persistent-dir $PERSISTENT `
>>   --runtime-dir $RUNTIME `
>>   --tif-path $TIF
[CraterML] Initialising DataHandler...
[CraterML] Local GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[CraterML] Cached GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[DataHandler] Resolving canonical ROI for tile 'ldem_80s_20m' with size 1024...
[DataHandler] Resolving GeoTIFF for tile 'ldem_80s_20m'...
[DataHandler] GeoTIFF found in runtime cache: C:\Endymion\runtime\LDEM_80S_20MPP_ADJ.TIF
[DataHandler] Opening GeoTIFF to read raster shape: C:\Endymion\runtime\LDEM_80S_20MPP_ADJ.TIF
[DataHandler] Raster shape resolved: (30400, 30400)
[DataHandler] Canonical ROI resolved: (14688, 15712, 14688, 15712)
[CraterML] Using ROI: (14688, 15712, 14688, 15712)
[CraterML] Derived dir: C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712
[CraterML] Building ML dataset from ROI cache...
[CraterML] Dataset rows: 1,048,576
[CraterML] Class counts: positive=551,659, negative=496,917
[CraterML] Train rows: 737,280
[CraterML] Test rows: 311,296
[CraterML] Train positive ratio: 0.504800
[CraterML] Test positive ratio: 0.576557
[CraterML] Training Random Forest baseline...
[CraterML] Held-out metrics: accuracy=0.7941, precision=0.8942, recall=0.7291, f1=0.8033
[CraterML] Predicting full ROI crater probabilities...
[CraterML] Saved probability raster: C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\crater_proba_ml_v1.npy
[CraterML] Saved model: C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\rf_crater_model_v1.joblib
[CraterML] Saved feature importances: C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\crater_proba_ml_v1_feature_importances.json
[CraterML] Saved summary: C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\crater_proba_ml_v1_summary.json
[CraterML] Done. Held-out metrics: accuracy=0.7941, precision=0.8942, recall=0.7291, f1=0.8033
```

App. Fig. N.1.2.1

```
>> python -m src.run_endymion `
>>   --persistent-dir $PERSISTENT `
>>   --runtime-dir $RUNTIME `
>>   --tif-path $TIF `
>>   --run-name app_t1_9 `
>>   --no-craters `
>>   --start 269 499 `
>>   --goal 122 558
[Endymion] Initialising run...
[Endymion] Persistent dir: C:\Endymion\persistent
[Endymion] Runtime dir: C:\Endymion\runtime
[Endymion] Local GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[Endymion] Cached GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[Endymion] Initialising DataHandler...
[Endymion] Resolving ROI...
[DataHandler] Resolving canonical ROI for tile 'ldem_80s_20m' with size 1024...
[DataHandler] Resolving GeoTIFF for tile 'ldem_80s_20m'...
[DataHandler] GeoTIFF found in runtime cache: C:\Endymion\runtime\LDEM_80S_20MPP_ADJ.TIF
[DataHandler] Opening GeoTIFF to read raster shape: C:\Endymion\runtime\LDEM_80S_20MPP_ADJ.TIF
[DataHandler] Raster shape resolved: (30400, 30400)
[DataHandler] Canonical ROI resolved: (14688, 15712, 14688, 15712)
[Endymion] Using ROI: (14688, 15712, 14688, 15712)
[Endymion] Loading or building derived terrain rasters...
[Endymion] Derived terrain ready in 0.01s
[Endymion] Skipping crater products (--no-craters enabled).
[Endymion] Building hazard map...
[Endymion] Hazard map ready in 0.03s
[Endymion] Building traversal cost grid...
[Endymion] Cost grid ready in 0.00s
[Endymion] Running pathfinder...
[Endymion] Pathfinder finished in 0.13s
[Endymion] Saving navigation outputs...
[Endymion] Run dir: C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_9
[Endymion] Running evaluator...
[Endymion] Metrics path: C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_9\metrics.json
[Endymion] Run complete.
====================================================================
Endymion run complete
====================================================================
Run dir      : C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_9
Derived dir  : C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712
ROI          : (14688, 15712, 14688, 15712)
Run name     : app_t1_9
Success      : True
Failure      : None
Total cost   : 636.4488525390625
Expansions   : 8807
--------------------------------------------------------------------
Path length m: 3510.7821048680194
Mean hazard  : 0.2659058178985311
Max hazard   : 0.4357956051826477
Safety score : 0.9883965566706623
Cost / m     : 0.16154634529362283
Metrics path : C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_9\metrics.json
```

App. Fig. N.1.2.2

```
>> python -m src.run_endymion `
>>   --persistent-dir $PERSISTENT `
>>   --runtime-dir $RUNTIME `
>>   --tif-path $TIF `
>>   --run-name app_t1_8 `
>>   --no-craters
[Endymion] Initialising run...
[Endymion] Persistent dir: C:\Endymion\persistent
[Endymion] Runtime dir: C:\Endymion\runtime
[Endymion] Local GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[Endymion] Cached GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[Endymion] Initialising DataHandler...
[Endymion] Resolving ROI...
[DataHandler] Resolving canonical ROI for tile 'ldem_80s_20m' with size 1024...
[DataHandler] Resolving GeoTIFF for tile 'ldem_80s_20m'...
[DataHandler] GeoTIFF found in runtime cache: C:\Endymion\runtime\LDEM_80S_20MPP_ADJ.TIF
[DataHandler] Opening GeoTIFF to read raster shape: C:\Endymion\runtime\LDEM_80S_20MPP_ADJ.TIF
[DataHandler] Raster shape resolved: (30400, 30400)
[DataHandler] Canonical ROI resolved: (14688, 15712, 14688, 15712)
[Endymion] Using ROI: (14688, 15712, 14688, 15712)
[Endymion] Loading or building derived terrain rasters...
[Endymion] Derived terrain ready in 0.01s
[Endymion] Skipping crater products (--no-craters enabled).
[Endymion] Building hazard map...
[Endymion] Hazard map ready in 0.03s
[Endymion] Building traversal cost grid...
[Endymion] Cost grid ready in 0.00s
[Endymion] Running pathfinder...
[Endymion] Pathfinder finished in 0.13s
[Endymion] Saving navigation outputs...
[Endymion] Run dir: C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_8
[Endymion] Running evaluator...
[Endymion] Metrics path: C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_8\metrics.json
[Endymion] Run complete.
====================================================================
Endymion run complete
====================================================================
Run dir      : C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_8
Derived dir  : C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712
ROI          : (14688, 15712, 14688, 15712)
Run name     : app_t1_8
Success      : True
Failure      : None
Total cost   : 636.4488525390625
Expansions   : 8807
--------------------------------------------------------------------
Path length m: 3510.7821048680194
Mean hazard  : 0.2659058178985311
Max hazard   : 0.4357956051826477
Safety score : 0.9883965566706623
Cost / m     : 0.16154634529362283
Metrics path : C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\navigation\app_t1_8\metrics.json
(.venv) PS C:\Users\vairc\Endymion\Endymion>
```

App. Fig. N.1.2.3

```
>> python -m src.run_endymion `
>>   --persistent-dir $PERSISTENT `
>>   --runtime-dir $RUNTIME `
>>   --tif-path $TIF `
>>   --run-name app_t3_10 `
>>   --no-craters `
>>   --start 2000 2000 `
>>   --goal 122 558
[Endymion] Initialising run...
[Endymion] Persistent dir: C:\Endymion\persistent
[Endymion] Runtime dir: C:\Endymion\runtime
[Endymion] Local GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[Endymion] Cached GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[Endymion] Initialising DataHandler...
[Endymion] Resolving ROI...
[DataHandler] Resolving canonical ROI for tile 'ldem_80s_20m' with size 1024...
[DataHandler] Resolving GeoTIFF for tile 'ldem_80s_20m'...
[DataHandler] GeoTIFF found in runtime cache: C:\Endymion\runtime\LDEM_80S_20MPP_ADJ.TIF
[DataHandler] Opening GeoTIFF to read raster shape: C:\Endymion\runtime\LDEM_80S_20MPP_ADJ.TIF
[DataHandler] Raster shape resolved: (30400, 30400)
[DataHandler] Canonical ROI resolved: (14688, 15712, 14688, 15712)
[Endymion] Using ROI: (14688, 15712, 14688, 15712)
[Endymion] Loading or building derived terrain rasters...
[Endymion] Derived terrain ready in 0.01s
[run_endymion] ERROR: ValueError: Start point (2000, 2000) is out of bounds for ROI shape (1024, 1024). Valid row range: 0 to 1023; valid col range: 0 to 1023. The resolved ROI is (14688, 15712, 14688, 15712). Coordinates must be ROI-local, not full-raster pixel c
oordinates
```

App. Fig. N.1.2.4

```
>> python -m src.run_crater_ml `
>>   --persistent-dir $PERSISTENT `
>>   --runtime-dir $RUNTIME `
>>   --tif-path $TIF
[CraterML] Initialising DataHandler...
[CraterML] Local GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[CraterML] Cached GeoTIFF: C:\Endymion\persistent\LDEM_80S_20MPP_ADJ.TIF
[DataHandler] Resolving canonical ROI for tile 'ldem_80s_20m' with size 1024...
[DataHandler] Resolving GeoTIFF for tile 'ldem_80s_20m'...
[DataHandler] GeoTIFF found in runtime cache: C:\Endymion\runtime\LDEM_80S_20MPP_ADJ.TIF
[DataHandler] Opening GeoTIFF to read raster shape: C:\Endymion\runtime\LDEM_80S_20MPP_ADJ.TIF
[DataHandler] Raster shape resolved: (30400, 30400)
[DataHandler] Canonical ROI resolved: (14688, 15712, 14688, 15712)
[CraterML] Using ROI: (14688, 15712, 14688, 15712)
[CraterML] Derived dir: C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712
[CraterML] Building ML dataset from ROI cache...
[CraterML] Dataset rows: 1,048,576
[CraterML] Class counts: positive=551,659, negative=496,917
[CraterML] Train rows: 737,280
[CraterML] Test rows: 311,296
[CraterML] Train positive ratio: 0.504800
[CraterML] Test positive ratio: 0.576557
[CraterML] Training Random Forest baseline...
[CraterML] Held-out metrics: accuracy=0.7941, precision=0.8942, recall=0.7291, f1=0.8033
[CraterML] Predicting full ROI crater probabilities...
[CraterML] Saved probability raster: C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\crater_proba_ml_v1.npy
[CraterML] Saved model: C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\rf_crater_model_v1.joblib
[CraterML] Saved feature importances: C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\crater_proba_ml_v1_feature_importances.json
[CraterML] Saved summary: C:\Endymion\persistent\derived\ldem_80s_20m\roi_14688_15712_14688_15712\crater_proba_ml_v1_summary.json
[CraterML] Done. Held-out metrics: accuracy=0.7941, precision=0.8942, recall=0.7291, f1=0.8033
```